

SimGolf (2002) — Guide de Référence Complet

Document fusionné à partir de 33 fichiers d'analyse (rétro-ingénierie de golf.exe + Terrain.dll + jgld.dll + sound.dll) Mai 2026 — Projet simgolf-re —
2900+ fonctions identifiées, 42 fichiers décompilés (C reconstruit), 5 736 assets documentés **Confiance** : ■ Confirmé (ASM/données) | ■■ Hypothèse |
■ Inconnu

Table des matières

- 1. [Architecture Globale](#)
- 2. [Structures de Données](#)
- 3. [Système de Terrain](#)
- 4. [Rendu Isométrique](#)
- 5. [Auto-Tiling & Textures](#) — 5.1→5.19 (groupes AE, orientations AD, variation cosmétique, bordures voisins, eau, overgrowth, multi-passes ASM, sub-tiling 4 quadrants) | ■ Décompilé
- 6. [Élévation du Terrain](#)
- 7. [Chemins \(Paths\)](#)
- 8. [Murs \(Walls\)](#)
- 9. [Éclairage \(Lighting\)](#)
- 10. [Guide de Port Phaser 4 / WebGL](#)
- 11. [Physique de Balle](#)
- 12. [IA des Golfeurs](#)
- 13. [Économie & Gestion](#)
- 14. [Scoring & Tournois](#)
- 15. [Interface Utilisateur](#)
- 16. [Audio & Animations](#)
- 17. [Sauvegarde & Formats de Fichier](#)
- 18. [Scénarios & Campagne](#)
- 19. [Arbres & Décorations](#)
- 20. [Guide de Réimplémentation \(condensé\)](#)
- 21. [Lacunes & Travaux Futurs](#)
- 22. [Annexes](#) — 17.1→17.8 (carte ASM, fichiers décompilés, outils)
- 23. [Pipeline d'Assets](#)

1. Architecture Globale

1.1 Binaires

golf.exe (946 Ko dépaqueté, 1.9 Mo packé)

■■■ .text	(754 Ko) – code exécutable, ~1900+ fonctions, base 0x00400000
■■■ .rdata	(288 Ko) – chaînes, vtable, tables de constantes
■■■ .data	(3.7 Mo) – données globales – section compressée (VRaw 152 Ko, VSize 3.6 Mo, ratio 24:1)
■■■ .rsrc	(4 Ko) – ressources Windows
■■■	3 sections packer residue

1.2 Dépendances

DLL	Rôle	Imports EXE
Terrain.dll	Rendu 3D isométrique (OpenGL 1.x, 45 imports actifs)	26 exports C++
jgld.dll	Moteur 2D GDI32 — sprites PCX, polices, framebuffer (libpng 1.0.5 intégré)	Export unique
sound.dll	Audio WAVE/MIDI (126 Ko, 12 exports)	12 exports
binkw32.dll	Cutscenes Bink Video	16 imports
WINMM	Windows Multimedia (audio bas niveau)	6 imports
KERNEL32	OS Core (77 imports)	77 imports
USER32	Fenêtrage (24 imports)	24 imports

■ **Preuve OpenGL actif** : 169+ appels IAT OpenGL vérifiés dans le .text. Le nom "JGL" dans les sources décompilées est un artefact de compilation — il résout vers OPENG32.dll. **Zéro** import DirectDraw.

1.3 Point d'Entrée et Initialisation

```
WinMain @ 0x004a682f
■■■■ HeapSetup()          @ 0x4a794d  - configure le tas
■■■■ SystemCheck()        @ 0x4a8dec  - vérifie OS (Windows 98/Me/XP) + résolution ≥ 800×600
■■■■ InitGameSystems()    @ 0x4a93ff  - init tous les sous-systèmes (1927 insn)
■■■■ GetModuleHandle()    @ thunk 0x4ba0bc
■■■■ CreateMainWindow()   @ ~0x4ab240 - crée la fenêtre Windows
■■■■ InitSound()          @ ~0x4aaff3  - initialise sound.dll
■■■■ InitGraphics()       @ ~0x4aaf3a  - initialise jgld.dll (JGL)
■■■■ InitGameState()      @ 0x4a50db  - charge données golfeurs + UI + écran titre (1894 insn)
■■■■ SetTimer()           - démarre la boucle de jeu (timer-driven, pas de PeekMessage)
■■■■ GameLoopCallback()   @ 0x4a5108  - première frame
■■■■ Message loop         @ 0x4aad6a
■■■■ CleanupAndExit()     @ 0x4a5119  - sortie
```

1.4 Boucle de Jeu

Le jeu n'a **pas** de boucle de messages classique (PeekMessage). Il utilise SetTimer/KillTimer :

```
SetTimer(hWnd, 1, interval, NULL) → timer Windows appelle GameLoopCallback()
```

```
GameLoopCallback @ 0x4a5108 (chaque frame, ~33ms / 30 FPS) :
■■■■ Traite les messages Windows (GetMessage/DispatchMessage)
■■■■ GameTickFunction @ 0x49846c - SIMULATION
■   ■■■■ 16 slots × 88 bytes (0x58)
■   ■■■■ WaitForMultipleObjects(2 slots) - timeout 20s
■   ■■■■ SEH protégé (__try/__except)
■   ■■■■ Appelle InputHandler si événement
■■■■ UI_Update() - met à jour l'interface
■■■■ Render()
    ■■■■ Terrain::render() - rendu OpenGL 3D isométrique
    ■■■■ JGL compositing - sprites 2D overlay (golfeurs, balles, UI)
    ■■■■ BitBlt/StretchBlt - présentation finale
```

1.5 Modèle Mémoire

Adresse	Taille	Contenu
0x008400b0	4 ptr	GameState* — pointeur vers l'objet principal du jeu
0x00842160	4	hInstance — handle d'instance
0x00840aac	4	HWND — handle fenêtre principale
0x00842040	0x480	InitPool — 32 slots × 0x24 bytes
0x0080d840	-0x960	TournamentObj array — 22 × 0x6c (108 bytes)
0x0083c340	256	StatusBuffer — barre d'état
0x0083afcc	4	CloseFlag — flag de fermeture
0x00840aa4	4	GameFlags — flags généraux
0x81ca10	88×N	BallData[] — tableau de structures balle

1.6 Gestion des Erreurs (SEH)

Le jeu utilise **Structured Exception Handling** (MSVC) :

```
mov ecx, dword ptr fs:[0] ; chaîne SEH
push ecx
mov dword ptr fs:[0], esp
; ... code protégé ...
mov ecx, dword ptr [esp + 0x2e8]
mov dword ptr fs:[0], ecx ; restaure chaîne SEH
```

Cela permet de catcher les access violations sans crasher.

2. Structures de Données

2.1 GameState (structure principale)

Pointeur global : [0x008400b0] → GameState*

Offset	Taille	Champ	Rôle
+0x00	4	gameTime	Temps de jeu écoulé

+0x04-0x24	—	cash, totalEarnings, dailyIncome, maintenanceCost, staffCost, greenFees, membershipFees, proShopSales, foodDrinks	Économie
+0x28	4	courseName	Pointeur nom du parcours
+0x2c	4	courseTheme	0=Parkland 1=Links 2=Desert 3=Tropical
+0x30	4	courseRating	Note SGA (0-100)
+0x34	4	holesCount	Nombre de trous construits
+0xf0	4	delayBetweenShots	Délai inter-tirs (5000/10000/20000/30000ms)
+0x170	0x58	slot[0]	Slot golfeur 0
+0x1c8	0x58	slot[1]	Slot golfeur 1
+0x77c	4	currentGolfer	ID golfeur actif
+0x780	4	previousGolfer	ID golfeur précédent

2.2 TickSlot (88 bytes = 0x58)

Stride entre slots : 0x58. 16 slots maximum.

```
interface TickSlot {
    golferId: number           // +0x00: ID du golfeur
    timer: number              // +0x04: timeGetTime() dernier tick
    state: number              // +0x08: 0=inactif, 1=actif, 2=terminé
    data: Uint8Array           // +0x09..0x57: données spécifiques
}
```

2.3 Tile (584 bytes = 0x248)

Schéma mémoire complet

Offset	Taille	Champ	

0x000	16	elevation[4]	← int32[4] (TL, TR, BR, BL)
0x010	4	waterLevel	← int32 (niveau d'eau)
0x014	4	flags/gridX	← int32 (flags généraux)
0x018	4	gridX	← int32 (position X grille)
0x01c	4	renderCategory	← int32 (0,1,2)
0x020	4	renderWidth	← int32 (largeur rendu px)
0x024	4	type	← int32 (TileType enum 0-15)
0x028	4	renderHeight	← int32 (hauteur rendu px)
0x02c	4	screenY	← int32 (coordonnée Y écran précalculée)
0x030	4	screenX	← int32 (coordonnée X écran précalculée)
0x034	16	renderObjects[4]	← void* C++ (pointeurs vers objets dynamiques)
0x044	4	renderPassCount	← int32 (nombre de passes de rendu)
0x048	4	pathConnections?	← int32 (non vérifié)
0x04c	4	gridY	← int32 (non vérifié)
0x050	12	normalX/Y/Z	← float[3] (normales)
0x05c	4	renderPassIndex?	← int32
0x060	12	[padding]	← alignement

0x06c	224	renderPasses[0..3]	← RenderPass[4] (stride 0x38)
0x14c	188	[inconnu]	← (inclut textureSlotA/B)

0x158	4	textureSlotA	← int32 (index ×12 vers table globale)
0x190	4	textureSlotB	← int32 (index ×12 vers table globale)

0x208	8	subObjectA	← struct (constructeur @ 0x1000f6e0)
0x210	36	subObjectB	← struct (constructeur @ 0x1000f7a0)

0x234	4	walls[4]	← uint8[4] (N/E/S/O)
0x238	12	[inconnu]	
0x244	1	dirtyFlag	← uint8
0x245	3	[padding]	
=====			
0x248	584	TOTAL	

TileType (enum, 16 valeurs)

ID	Nom	Description	ID	Nom	Description
----	-----	-------------	----	-----	-------------

0	Rough	Herbe haute (naturelle)	8	GrassySand	Transition sable→herbe
1	Fairway	Herbe tondue	9	GrassBunker	Transition herbe→sable
2	PuttingGreen	Green	10	Tee	Départ
3	SandBunker	Bunker de sable	11	Cliff	Falaise
4	WaterShallow	Eau peu profonde	12	Path	Chemin
5	WaterMiddle	Eau moyenne	13	Building	Bâtiment
6	WaterDeep	Eau profonde	14	Tree	Arbre
7	DeepRough	Herbe très haute (hors-limites)	15	Flower	Fleur/Buisson

RenderPass (56 bytes = 0x38)

Offset	Taille	Champ	Description
+0x00	4	textureID	ID texture OpenGL/JGL
+0x04	4	texCoord	Coordonnée texture
+0x08	4	field_08	(copié depuis passe précédente)
+0x0c	4	field_0c	(copié vers passe suivante)
+0x10	4	field_10	Données rendu
+0x14	4	field_14	(inter-passe)
+0x18	4	field_18	(inter-passe)
+0x1c	4	field_1c	(inter-passe)
+0x20	4	field_20	Coordonnée rendu
+0x24	4	field_24	Coordonnée rendu
+0x28	4	field_28	Données rendu
+0x2c	4	field_2c	(depuis passe suivante)
+0x30	4	field_30	Init = 0
+0x34	4	field_34	Init = 1.0 (float)

2.4 Golfer (.pro / .glf)

```
interface Golfer {
    id: number           // Identifiant unique
    name: string          // Nom complet
    isPro: boolean        // Pro ou célébrité ?
    skills: {             // Compétences (0-10)
        putting: number
        power: number
        accuracy: number
        imagination: number
        timing: number
        luck: number
    }
    money: number          // Argent gagné
    morale: number         // Moral (0-100)
    stamina: number        // Endurance
    fatigue: number        // Fatigue
    tournamentsWon: number // Tournois gagnés
    bestScore: number      // Meilleur score
}
```

Les golfeurs pros sont stockés dans des fichiers .pro (binaires, 1008 bytes + portrait PCX). Les célébrités dans .chr. Les golfeurs personnalisés dans .glf.

2.5 TournamentObj (108 bytes = 0x6c)

Tableau à l'adresse 0x80d840, stride 0x6c, 22 entrées max.

```
interface TournamentDef {
    name: string          // +0x00: Nom (pointeur .rdata)
    prizePool: number      // +0x04: Cagnotte ($)
    entryFee: number       // +0x08: Frais d'inscription
    minHoles: number       // +0x0c: Trous minimum requis
    prestigeReq: number    // +0x10: Prestige requis
    prestigeGain: number   // +0x14: Prestige gagné
    fieldSize: number      // +0x18: Participants max
    courseTheme: number    // +0x1c: Thème requis (-1=tous)
    sgaRequired: number    // +0x20: Score SGA min (0-100)
    tier: number            // +0x24: Niveau (1-10)
}
```

2.6 BallState (88 bytes @ 0x81ca10)

```
interface BallState {
    positionX: number    // +0x00: Position X terrain
    positionY: number    // +0x04: Position Y terrain
    positionZ: number    // +0x08: Hauteur Z
    velocityX: number    // +0x0c: Vitesse X
    velocityY: number    // +0x10: Vitesse Y
    velocityZ: number    // +0x14: Vitesse Z
    spinX: number        // +0x18: Effet Magnus X
    spinY: number        // +0x1c: Effet Magnus Y
    spinZ: number        // +0x20: Effet Magnus Z
    clubId: number       // +0x24: ID du club utilisé (0-13)
    lieType: number      // +0x28: Type de sol (0=Tee..5=Water)
    // +0x2c-0x57: flags d'état, timer, padding
}
```

2.7 CourseTheme (enum)

```
enum CourseTheme { Parkland = 0, Links = 1, Desert = 2, Tropical = 3 }
```

3. Système de Terrain

3.1 Architecture

Le terrain est géré par **Terrain.dll** (972 Ko, singleton, getInstance @ 0x100031a0).

Terrain (taille ~1.4 Mo = 0x164AD0)
 ■■■ TileGrid (grille de tuiles) – dispatch hub @ 0x485e80
 ■■■ Textures – 4 thèmes × textures BMP (~2500 fichiers)
 ■■■ Élévation – 4 coins par tuile (int32[4], niveaux 0-4)
 ■■■ Chemins – splines Bézier/Cardinal
 ■■■ Normales – float[3] pour éclairage

3.2 Grille de Tuiles

- **Dimensions** : configurable (resize @ 0x10009ed0), par défaut 64×64
- **Tableau row-major** : tiles[y * width + x]
- **Accès** : tileAt(x, y) @ 0x10001d50 — bounds check + base + index × 0x248
- **Début tableau** : Terrain + 0x3A4
- **Preuve sizeof(Tile)=584** : imul eax, eax, 0x248 dans le désassemblage

3.3 Élévation

Chaque tuile a **4 coins** (TL, TR, BR, BL). **Plage** : **0-4** (5 niveaux).

Niveau	Hauteur	Usage
0	Base	Terrain plat
1	Faible	Petite bosse
2	Moyen	Colline
3	Haut	Haute colline
4	Max	Point culminant

Fonctions : -getElevation(tile, corner) @ 0x10001de0 -elevateCorner(tile, corner) @ 0x1000a5c0 -lowerCorner(tile, corner) @ 0x1000a620 -lowerEdgeCorner(tile, ...) @ 0x10001154

Contrainte : Déclivité maximale de **1 niveau** entre coins adjacents (vérifié par setType()).

Propagation récursive : Quand un coin est surélevé, les coins partagés des tuiles adjacentes sont mis à jour pour éviter les déchirures visuelles.

3.4 Vérité fondamentale : le terrain initial est 100% EAU

Après resetTerrain() (0x1000aa10), chaque tuile a :

```
tile.type = TileType.WaterShallow (4) – TOUT EST DE L'EAU
tile.elevation = [0, 0, 0, 0] – parfaitement plat
tile.screenX = colonne
tile.screenY = ligne
tile.renderObjectPtrs = [null, null, null, null]
tile.flags = 0
tile.renderCategory = 0
```

Le joueur doit peindre les types de terrain, sculpter les hauteurs et placer les objets.

3.5 Murs

Chaque tuile peut avoir jusqu'à **4 murs** (N/E/S/O, booléens). Stockés à offset +0x234 (uint8[4]). - getWall(tile, side) @ 0x10001e80 - setWall(tile, type, side, bool) @ 0x1000a560

Utilisés pour : falaises, bâtiments, bords d'eau, limites du terrain.

3.6 Types de Terrain — Familles de Voisinage

Pour l'auto-tiling, les types sont regroupés en familles pour éviter les fausses bordures :

```
grass family    = GRASS(0), TREE(14), FLOWER(15), ROUGH(7)
play family    = FAIRWAY(1), TEE(10), GREEN(2)
sand family    = SANDBUNKER(3), GRASSY_SAND(8), GRASS_BUNKER(9)
water family   = WATERSHALLOW(4), WATERMIDDLE(5), WATERDEEP(6)
path family    = PATH(12), BRIDGE
building       = BUILDING(13)
```

Deux tuiles de la même famille ne produisent **pas** de bordure de transition.

4. Rendu Isométrique

4.1 Dual-Moteur Graphique

```

■      jgld.dll – Moteur 2D (GDI32 pur)      ■
■  Sprites UI (PCX), polices, framebuffer DIBSection,
■  compositing final BitBlt/StretchBlt      ■
■
■      Terrain.dll – Moteur 3D (OpenGL 1.x)  ■
■  Terrain isométrique, 45 imports OpenGL actifs ■
■  glBegin/glEnd, glVertex3f, glTexCoord2f, ... ■

```

4.2 Projection Dimétrique (2:1)

Ce n'est pas une vraie projection isométrique (120°) mais **dimétrique** (rapport 2:1, angle $\approx 26.565^\circ$).

```
TILE_W = 128 pixels
TILE_H = 64 pixels
```

```
World → Screen :
screenX = (mapX - mapY) × 64      // (TILE_W / 2)
screenY = (mapX + mapY) × 32      // (TILE_H / 2) - (elevation × offset)
```

```
Screen → World (picking) :
    relX = screenX - cameraOffsetX
    relY = screenY - cameraOffsetY
    mapX = (relX / 64 + relY / 32) / 2
    mapY = (relY / 32 - relX / 64) / 2
    tileX = floor(mapX)
    tileY = floor(mapY)
```

4.3 Triangle Layout (Choix de la Diagonale)

Chaque tuile carrée est divisée en **2 triangles**. Le choix de la diagonale est critique :

```
int getTriangleLayout(float hTL, float hTR, float hBR, float hBL) {
    float d1 = fabsf(hTL - hBR); // diagonale TL↔BR
    float d2 = fabsf(hTR - hBL); // diagonale TR↔BL
    return (d1 < d2) ? 0 : 1;    // 0 = TL-BR, 1 = TR-BL
}
```

Règle : La diagonale relie les 2 coins ayant la plus petite différence de hauteur.

4.4 Pipeline de Rendu OpenGL

```

Terrain::render() @ 0x10005990
■■■ glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
■■■ glLoadIdentity / glPushMatrix
■■■ Calcul matrice vue (rotation FOV, angle 45° iso)
■■■ localRender() – frustum culling logiciel
■ ■■■ glTranslatef (centrage sur tuile caméra)
■ ■■■ Boucle 5x5 autour de la caméra (selon angle : 0°/90°/180°/270°)
■ ■ ■■■ renderSingleTile(tile) @ 0x1000e6c0
■ ■ ■■■ for pass ∈ renderPasses :
■ ■ ■■■ glBindTexture(textureID)
■ ■ ■■■ glBegin(GL_TRIANGLES)
■ ■ ■■■ glTexCoord2f + glVertex3f × 3
■ ■ ■■■ glEnd()
■ ■ ■■■ overlay (si visible)
■ ■ ■■■ post-process + effets
■ ■■■ renderTileDecorations() – arbres, bâtiments
■■■ Itérateur global tiles (visiteur)
■■■ glPopMatrix
■■■ SwapBuffers(hDC)

```

4.5 Interpolation de Hauteur (Barycentrique)

```

function interpolateHeight(tile: Tile, localX: number, localY: number): number {
  const el = tile.elevation; // [TL, TR, BR, BL]
  const b0 = Math.abs(el[0] - el[2]) < Math.abs(el[1] - el[3]) ? 0 : 1;

  if (b0 === 0) { // Diagonale TL-BR
    if (localX + localY > 1) { // Triangle BR
      return (localX+localY-1)*el[2] + (1-localX)*el[3] + (1-localY)*el[1];
    } else { // Triangle TL
      return (1-localX-localY)*el[0] + localX*el[1] + localY*el[3];
    }
  } else { // Diagonale TR-BL – rotation 90°
    const lx_ = localY, ly_ = 1 - localX;
    if (lx_ + ly_ > 1) {
      return (lx_+ly_-1)*el[2] + (1-lx_)*el[1] + (1-ly_)*el[3];
    } else {
      return (1-lx_-ly_)*el[0] + lx_*el[1] + ly_*el[3];
    }
  }
}

```

4.6 Calcul des Normales

```

function calcTileNormal(tile: Tile): Vec3 {
  const el = tile.elevation;
  const v1 = { x: 1, y: 0, z: el[1] - el[0] }; // TL → TR
  const v2 = { x: 1, y: 1, z: el[2] - el[0] }; // TL → BR
  const v3 = { x: 0, y: 1, z: el[3] - el[0] }; // TL → BL
  const n = normalize(add(cross(v1, v2), cross(v2, v3)));
  return n;
}

```

calcAllNormals() @ 0x1000a740 recalcule les normales dans un rayon autour d'une tuile pour garantir la continuité de l'éclairage.

4.7 Caméra et Zoom

Le système de caméra gère 4 angles de vue (0°, 90°, 180°, 270°) avec un zoom variable.

Zoom : setZoomLevel(level) @ 0x10008fc0 — 4 niveaux : ×0.5, ×1, ×2 **Animation** : SmoothInterpolator @ 0x4969e0 — animation fluide (~300 insns)
FOV : Ajusté selon résolution : - 800×600 → 38.86° - 1024×768 → 40.58° - 1280×1024 → 40.83°

Décompilation de localRender — Gestion de la rotation caméra :

```

// Terrain::localRender @ 0x1000117c
// Rendu avec frustum culling basé sur l'angle
void __thiscall Terrain::localRender(Terrain* this, Tile* centerTile,
                                     Tile* cameraTile, float angle) {
  glLoadIdentity();
  glPushMatrix();

```

```

// Rotation FOV (axe Y)
glRotated(g_fovY, 0, 1, 0, 0, 0, 0);
// Inclinaison de la vue
glRotatef(g_angleOffset + angle, 0, 1, 0);

// Centrage sur la tuile caméra
if (cameraTile != NULL) {
    int tileZ = getTileZ(cameraTile);    // Coord Z dans la grille
    int tileX = getTileX(cameraTile);    // Coord X dans la grille
    float zoom = (float)(19 - tileZ) * g_zoomFactor;
    float x = (float)(19 - tileX) * g_zoomFactor;
    glTranslatef(x, 0, zoom);
}

// Éclairage selon angle
setupLighting(angle);

// Les 4 angles déterminent l'ordre de rendu (painter's algorithm)
// pour éviter les artefacts de profondeur :
if (angle == 0.0f) {    // Vue Nord
    for (int z = -2; z <= 2; z++)
        for (int x = 2; x >= -2; x--)
            renderTile(tileAt(this, cx+x, cz+z));
}
else if (angle == 90.0f) {    // Vue Est
    for (int x = 2; x >= -2; x--)
        for (int z = -2; z <= 2; z++)
            renderTile(tileAt(this, cx+x, cz+z));
}
else if (angle == 180.0f) {    // Vue Sud
    for (int z = -2; z <= 2; z++)
        for (int x = -2; x <= 2; x++)
            renderTile(tileAt(this, cx+x, cz+z));
}
else {    // Vue Ouest (défaut)
    for (int z = -2; z <= 2; z++)
        for (int x = -2; x <= 2; x++)
            renderTile(tileAt(this, cx+x, cz+z));
}

// Itérateur global – rend les tuiles non visitées
Iterator it = createIterator(&this->tileIterator);
while (hasNext(it)) {
    renderSingleTile((Tile*)next(it));
    step(it);
}
destroy(it);

glPopMatrix();
glFlush();
}

```

Fonctions de coordonnées tuile :

```

int getTileX(Tile* tile); // @ 0x10005960 – retourne coordonnée X
int getTileZ(Tile* tile); // @ 0x10006810 – retourne coordonnée Z (=Y grille)
Tile* tileAt(Terrain*, int x, int z); // @ 0x1000108c – accès grille

```

Variables globales caméra :

```

extern float _DAT_1005f340; // Facteur de zoom (translate)
extern float _DAT_1005f344; // Offset d'angle de vue
extern float _DAT_10070a10; // FOV axe Y (38.86–40.83 selon résolution)

```

4.8 jgld.dll — Moteur 2D

	Propriété	Valeur
Fichier		jgld.dll
Taille		396 Ko (packé)

Export unique
Fonctions

get_graphsy_object_ptr → JackalClass singleton
~1200 (debug build)

JackalClass (332 bytes = 0x14C) :

```
JackalClass {  
    vtable, hWnd, hDC, hSpriteDC, hSpriteBMP (DIBSection),  
    hPalette, screenWidth, screenHeight, ...  
}
```

Pipeline : Créer DIBSection → dessiner sprites PCX → TextOut polices → BitBlt/StretchBlt final.

Transparence : Fichiers _alpha.pcx = masque, compositing via SRCAND + SRCPAINT (technique GDI classique).

5. Auto-Tiling & Textures

5.1 Les 4 Axes de Variation

Chaque tuile est définie par **4 axes orthogonaux** qui déterminent son apparence :

Axe 1 : Forme géométrique → A(plat), B(pente), C(coin), D(diagonale), E(raide)
Axe 2 : Orientation de bordure → A(Nord), B(Est), C(Sud), D(Ouest)
Axe 3 : Cosmétique → 0001-0009 (anti-répétition)
Axe 4 : Multi-passes → jusqu'à 4 textures superposées

5.2 Groupes Géométriques (A–E)

Déterminés par les 4 hauteurs des coins :

Groupe	Condition	Nb formes	Exemple [TL, TR, BR, BL]
A	Plat : tous égaux	1	[0, 0, 0, 0]
B	Pente simple : 2 coins adjacents	4	[1, 1, 0, 0]
C	Coin : 1 coin différent	8	[1, 0, 0, 0]
D	Diagonale : coins opposés	2	[1, 0, 1, 0]
E	Raide : $\Delta \geq 2$ entre adjacents	4+	[2, 2, 0, 0]

5.3 Auto-Tiling (Masque de Voisinage)

computeNeighborMask(tile, x, y, grid) :

- Pour chaque voisin cardinal N(1), E(2), S(4), W(8) :
 - Si type différent ET pas dans la même famille → marquer le bit
- Priorité : $N > E > S > W$ (première direction active = suffixe A-D)
- Mapper le bit prioritaire :
 $N \rightarrow A$ (bord Nord), $E \rightarrow B$ (Est), $S \rightarrow C$ (Sud), $W \rightarrow D$ (Ouest)

5.4 Anti-Répétition (Variation Cosmétique)

```
// Compteur global par type de terrain (incrémental)  
function selectCosmeticVariation(type: TileType): number {  
    const maxVar = typeInfo[type].maxVariation;  
    if (maxVar <= 0) return 0;  
    globalCounters[type] = (globalCounters[type] + 1) % maxVar;  
    return globalCounters[type];  
}
```

Valeurs de maxVariation constatées :

Type	Desert	Parkland	Links	Tropical
Rough	9	5	5	4
Fairway	5	5	5	5
Green	5	5	5	5
Tee	25	25	25	25
WaterShallow	9	9	9	9
SandBunker	5	5	5	5

5.5 Convention de Nommage des Textures

{TypeTerrain}{Groupe/Orientation}{Variation à 4 chiffres}.bmp

Exemples :

```

RoughA0001.bmp      → Herbe, forme A (plate), variante 1
RoughB0003.bmp      → Herbe, forme B (pente), variante 3
GrassySandB0002.bmp → Bordure sable/herbe côté Est, variante 2
TeeA0025.bmp        → Départ, 25e variante

```

5.6 Table Globale des Textures (0x100687f8)

```
g_textureTable[type][orientation][variation] = GLuint (handle texture OpenGL)
```

```

Chaque type      : 0x384 = 900 bytes
Chaque orient.  : 0x024 = 36 bytes
Chaque variat.  : 4 bytes (GLuint)

```

Calcul adresse : 0x100687f8 + type×0x384 + orientation×0x24 + variation×4

Population faite par loadNewCourseType() (0x10001af0) au changement de thème.

5.7 Nombre de Textures par Thème

Thème	Fichiers BMP
Desert	732 (le plus complet)
Links	644
Parkland	602
Tropical	547 (le moins complet)

5.8 Mécanisme de Génération des Bordures — Analyse Complète

Le système de bordures entre tuiles hétérogènes repose sur **4 mécanismes distincts** identifiés dans Terrain.dll :

5.8.1 Stockage des Voisins (Auto-Tiling Loop @ 0x1000a130)

La fonction d'initialisation du rendu (updateAllTileNeighbors @ 0x1000a130, 549 bytes, 174 insn) effectue **deux passes** sur toute la grille :

```

; === PASSE 1 : Initialisation (0x1000a16a-0x1000a207) ===
; Pour chaque (x, y) dans la grille :
;   tileInit(x, y)                → tile par défaut (type=WaterShallow)
;   getType(tile)                 → lit le type
;   [0x100010c3 → 0x10002060]     → compute render passes

; === PASSE 2 : Stockage des voisins (0x1000a20e-0x1000a342) ===
; Pour chaque (x, y) dans la grille :
;
;   Voisin Nord  (x,  y-1) → tileAt(x,  y-1)  push 2 → renderObjects[2]
;   Voisin Ouest (x-1, y)  → tileAt(x-1, y)    push 0 → renderObjects[0]
;   Voisin Sud   (x,  y+1) → tileAt(x,  y+1)    push 3 → renderObjects[3]
;   Voisin Est   (x+1, y)  → tileAt(x+1, y)    push 1 → renderObjects[1]
;
; Side mapping : 0=Ouest, 1=Est, 2=Nord, 3=Sud
; La fonction [0x10001113 → 0x1000c520] stocke le ptr voisin :
;   this->renderObjects[side] = neighborTile

```

5.8.2 Le Rôle de renderObjects[4] (offset +0x34)

Chaque Tile stocke les pointeurs de ses 4 voisins dans tile->renderObjects[0..3] (offset +0x34, 4 × 4 = 16 bytes). Ces pointeurs sont utilisés au moment du rendu pour comparer les types :

```

// Structure Tile – extrait des champs critiques
// Offset  Taille  Champ                Rôle
// -----
// 0x034   16      renderObjects[4] Pointeurs C++ vers les 4 voisins
//                                     [0]=Ouest, [1]=Est, [2]=Nord, [3]=Sud
// 0x044    4      renderPassCount  Nombre de passes de rendu (typique 1-3)
// 0x06c   var.    renderPasses[]   Tableau des passes de rendu
// 0x240    4      textureOffset    Variation cosmétique (0..maxVariation-1)

```

5.8.3 Le Système de Familles (typeInfo@this+0x40)

La table typeInfo à this + 0x40 contient les métadonnées de chaque type de terrain : - Stride : **24 bytes** par type (0x18) - Champs identifiés : - +0x00 : **maxVariation** (int32) — utilisé par setType pour rand() % maxVariation - +0x04 : **family** (int32) — groupe de voisinage (0=grass, 1=play, 2=sand, 3=water, etc.) - +0x08 : **renderMode** (int32) — 0=élévation, 1=bordure, 2=objet3D - +0x0c : **borderOverride** (int32) — type de texture de bordure à utiliser (ex: GrassBunker=9, GrassySand=8) - +0x10-0x17 : flags, seuils, limites d'élévation

Mapping des familles (complet, vérifié par analyse ASM + fichiers réels) :

Famille	ID	Types membres
grass	0	Rough(0), DeepRough(7), Brush, Woods(14), Flower(15), Natural, Marsh, Vegetation, Overgrowth(18) ■■, Rock(16)
play	1	Fairway(1), Tee(10), Green(2), FirmFairway(19), TrickyGreen(22)
sand	2	SandBunker(3), GrassySand(8), GrassBunker(9), PotSandBunker(20), ZenSand(21)
water	3	WaterShallow(4), WaterMiddle(5), WaterDeep(6)
path	4	Path(12), Bridge(23), Ravine(24)
building	5	Building(13), RetainingWall(25)
cliff	6	Cliff(11)
rock	7	Rock(16) — peut être grass(0) selon le build

■■ Overgrowth est le SEUL type grass qui possède des textures A-D directionnelles (renderMode=1). Tous les autres types grass utilisent A-E uniquement pour la géométrie d'élévation.

Les 3 Mécanismes de Génération de Bordure

Le jeu original utilise la table typeInfo (stride 24 bytes, 30+ entrées) pour déterminer si un type génère des bordures. Trois cas possibles :

1. Auto-bordure (renderMode=1) : Le type a ses propres textures A-D qui sont des bordures directionnelles.

Type	Famille	Textures	Génère bordure vers
WaterShallow(4)	water	A-D × 9	27 types (tous sauf water)
WaterMiddle(5)	water	A-D × 9	27 types
WaterDeep(6)	water	A-D × 5	27 types
Cliff(11)	cliff	A-D × 9	29 types (tous)
GrassBunker(9)	sand	A-D × 9	24 types (tous sauf sand)
GrassySand(8)	sand	A-D (par thème)	24 types
Overgrowth(18)	grass ■■	A-D × 9	19 types (tous sauf grass)
Ravine(24)	path	A-D × 9	23 types (tous sauf path)

2. borderOverride (borderOverride ≠ -1) : Le type n'a pas ses propres textures A-D, mais utilise celles d'un autre type via le champ borderOverride (offset +0x0c dans typeInfo).

Type	borderOverride	Utilise les textures de	Pourquoi
SandBunker(3)	9	GrassBunker	Le sable du bunker borde l'herbe via GrassBunker A-D (brun)
PotSandBunker(20)	9	GrassBunker	Idem pour les pot bunkers
Fairway(1)	8	GrassySand	Le fairway borde le rough via GrassySand A-D (vert)
FirmFairway(19)	8	GrassySand	Idem pour le fairway dur

3. Seam (borderOverride = -1) : Le type ne génère aucune bordure. Les textures adjacentes se rencontrent directement.

Type	Famille	Pourquoi pas de bordure
Rough(0), DeepRough(7)	grass	A-E = géométrie, pas bordures
Woods(14), Brush, Rock(16)	grass	Idem
PuttingGreen(2), Tee(10)	play	Types plats, pas de textures A-D
Marsh(17), Natural, Vegetation	grass	Types de remplissage, groupe A seulement
Path(12), Bridge(23)	path	Types plats
Building(13), RetainingWall(25)	building	Types bâtiment
ZenSand(21)	sand	Sable décoratif plat

5.8.4 Matrice Exhaustive des Transitions entre Types de Terrain

Règle fondamentale : Une bordure apparaît **uniquement** quand le type adjacent a une texture de bordure dans son set de fichiers BMP (groupes A–D). La bordure est **monodirectionnelle** — portée par le type qui a une texture de bordure, pas par celui qui ne l'a pas.

3 comportements possibles : - ■ Même famille → pas de bordure, les types se fondent visuellement - ■ Seam → familles différentes mais aucun type n'a de texture de bordure, jonction nette - ■ Bordure → le type qui a une texture A–D génère la bordure sur le côté adjacent

Types avec textures de bordure (A–D) : WaterShallow, WaterMiddle, WaterDeep, Cliff, GrassySand, GrassBunker

Types sans bordure (A–E ou A) : tous les autres (Rough, Fairway, Green, Tee, SandBunker, Path, Building, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall)

5.8.4.1 Rough (grass, 29 types adjacents)

Catégorie	Résultat	Voisins
■ Même famille (10)	Pas de bordure	Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Bordure (6)	Le type eau/cliff/sable porte la bordure	Fairway, Green, Tee, SandBunker, Path, Building, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall
■ Seam (13)	Jonction directe, pas de transition	

5.8.4.2 Woods (grass, 29 types adjacents)

Catégorie	Résultat	Voisins
■ Même famille (10)	Pas de bordure	Rough, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Bordure (6)	Le type eau/cliff/sable porte	Fairway, Green, Tee, SandBunker, Path, Building, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall
■ Seam (13)	Jonction directe	

5.8.4.3 DeepRough (grass, 29 types adjacents)

Catégorie	Résultat	Voisins
■ Même famille (10)	Pas de bordure	Rough, Woods, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Bordure (6)	Le type eau/cliff/sable porte	Fairway, Green, Tee, SandBunker, Path, Building, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall
■ Seam (13)	Jonction directe	

5.8.4.4 Fairway / Green / Tee / FirmFairway (play, 29 types adjacents chacun)

Catégorie	Résultat	Voisins
■ Même famille (3)	Pas de bordure	entre eux (Fairway↔Green, Fairway↔Tee, Fairway↔FirmFairway, Green↔Tee, Green↔FirmFairway, Tee↔FirmFairway) WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Bordure (6)	Le type eau/cliff/sable porte	Rough, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, SandBunker, Path, Building, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall
■ Seam (20)	Jonction directe, limite visible entre fairway et rough/grass	

■■ Fairway → Rough = Seam : c'est la transition la plus fréquente sur un vrai parcours. Le fairway tondu rencontre le rough directement — il n'y a aucune texture de transition, juste une limite visuelle nette entre les deux textures de base.

5.8.4.5 WaterShallow / WaterMiddle / WaterDeep (water, 29 types adjacents chacun)

Catégorie	Résultat	Voisins
■ Même famille (2)	Pas de bordure	WaterShallow↔WaterMiddle, WaterShallow↔WaterDeep, WaterMiddle↔WaterDeep
■ Bordure (27)	L'eau porte sa bordure vers tous les autres types	Rough, Fairway, Green, Tee, SandBunker, Cliff, Path, Building, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, GrassySand, GrassBunker, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall — chaque voisin affiche Water{A-D} sur le côté adjacent à l'eau
■ Seam (0)	Aucun — l'eau génère toujours une bordure	—

■ L'eau est le type le plus connectif : elle génère une bordure vers 27 types sur 29. Seuls les autres types d'eau (même famille) sont exempts.

5.8.4.6 Cliff (cliff, 29 types adjacents)

Catégorie	Résultat	Voisins
■ Même famille (0)	Cliff est seul dans sa famille	—
■ Bordure (29)	La falaise porte sa bordure vers tous les autres types	Rough, Fairway, Green, Tee, WaterShallow, WaterMiddle, WaterDeep, SandBunker, Path, Building, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, GrassySand, GrassBunker, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall
■ Seam (0)	Aucun	—

■ Cliff est le type le plus connectif : il génère une bordure vers les 29 autres types. C'est normal pour une falaise qui peut borde n'importe quel terrain.

5.8.4.7 SandBunker / PotBunker / ZenSand / PotSandBunker (sand, 29 types adjacents chacun)

Catégorie	Résultat	Voisins
■ Même famille (5)	Pas de bordure entre bunkers	entre eux (SandBunker↔PotBunker, SandBunker↔ZenSand, SandBunker↔PotSandBunker, etc.) + GrassySand, GrassBunker
■ Bordure (4)	Le type eau/cliff porte, ou les types sable de transition	WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D
■ Seam (20)	Jonction directe avec grass/play/path	Rough, Fairway, Green, Tee, Path, Building, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, FirmFairway, Bridge, Ravine, RetainingWall

■■ SandBunker → Rough = Seam : le sable du bunker et le rough se rencontrent directement. Pour les bunkers adjacents au rough, la bordure vient du côté rough uniquement (GrassBunker) — mais si le rough est le type de base, il n'a pas de bordure, donc le seam est visible.

5.8.4.8 GrassySand / GrassBunker (sand, 29 types adjacents chacun)

Catégorie	Résultat	Voisins
■ Même famille (5)	Pas de bordure	SandBunker, PotBunker, ZenSand, PotSandBunker, et entre GrassySand↔GrassBunker
■ Bordure (24)	Le type de transition sable génère vers la majorité des types	Vers tous sauf les 5 sand + Bridge, Ravine (même famille path)
■ Seam (0)	Aucun	—

■ GrassySand et GrassBunker sont des types de bordure spécialisés : ils n'ont pas de seam car ils existent uniquement pour générer des transitions. GrassySand borde le côté sable (il montre l'herbe dans le sable), GrassBunker borde le côté herbe (il montre le sable dans l'herbe).

5.8.4.9 Path / Bridge / Ravine (path, 29 types adjacents chacun)

Catégorie	Résultat	Voisins
■ Même famille (2)	Pas de bordure	Path↔Bridge, Path↔Ravine, Bridge↔Ravine

■ Bordure (6)	Le type eau/cliff/sable porte	WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Seam (21)	Jonction directe	Tous les autres types (grass, play, building, decoration)

5.8.4.10 Building (building, 29 types adjacents)

Catégorie	Résultat	Voisins
■ Même famille (0)	Building seul dans sa famille	— WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Bordure (6)	Le type eau/cliff/sable porte	Rough, Fairway, Green, Tee, SandBunker, Path, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall
■ Seam (23)	Jonction directe avec grass/play/path/decor	

5.8.4.11 RetainingWall (decoration, 29 types adjacents)

Catégorie	Résultat	Voisins
■ Même famille (0)	Seul dans sa famille	— WaterShallow → WaterShallowA-D , WaterMiddle → WaterMiddleA-D , WaterDeep → WaterDeepA-D , Cliff → CliffA-D , GrassySand → GrassySandA-D , GrassBunker → GrassBunkerA-D
■ Bordure (6)	Le type eau/cliff/sable porte	Tous les autres types
■ Seam (23)	Jonction directe	

5.8.4.12 Résumé : statistiques de connectivité

Type	Bordures possibles	Même famille	Seam possibles
Cliff	29 (max)	0	0
Water (les 3)	27	2	0
GrassySand / GrassBunker	24	5	0
Rough / Woods / DeepRough / Brush / Flower / Natural / Rock / Marsh / Vegetation / Overgrowth / Flowerbed	6	10	13
Fairway / Green / Tee / FirmFairway	6	3	20
SandBunker / PotBunker / ZenSand / PotSandBunker	4	5	20
Path / Bridge / Ravine	6	2	21
Building / RetainingWall	6	0	23

5.8.4.13 Les réponses aux questions fréquentes

Fairway → Rough : ■ Seam — pas de bordure. Les deux textures de base se rencontrent directement. Le fairway (play) et le rough (grass) sont dans des familles différentes, mais ni l'un ni l'autre n'a de texture de bordure. Résultat : une limite visuelle nette entre le gazon tondu et l'herbe haute.

Woods → Rough : ■ Pas de bordure — même famille (grass). Woods et Rough se fondent visuellement. La distinction vient uniquement du motif de texture différent (WoodsA0001 vs RoughA0001).

Fairway → SandBunker : ■ Double bordure — GrassBunker{A-D} côté fairway + GrassySand{A-D} côté bunker. Les deux types de transition sont générés simultanément.

WaterShallow → WaterMiddle : ■ Pas de bordure — même famille (water). Les deux eaux se rencontrent sans transition visible. C'est pourquoi les plans d'eau de profondeur variable ont un aspect naturel (même texture, profondeur variable dans les shaders).

5.8.5 Cas Particuliers (vraies exceptions non couvertes par la matrice)

Multi-pass rendering : Une tuile peut avoir jusqu'à 4 renderPasses. Dans le cas d'un carrefour où 4 types différents se rencontrent, chaque côté peut avoir une texture de bordure différente. La hiérarchie de priorité est : Nord > Est > Sud > Ouest (side 2 > 1 > 3 > 0).

RoughE (élévation raide) : Le groupe E ($\Delta \geq 2$) n'existe que pour certains types (Rough, DeepRough, Natural, Rock, Building). La lettre E signifie toujours élévation, pas orientation. Woods, Water, GrassySand n'ont **jamais** de groupe E.

SandBunker 1A-4A : Le préfixe numérique (1-4) code la **forme du bunker** (rond, ovale, long, carré), pas l'orientation. Contrairement à A-D pour les autres types de bordure, le préfixe n'est pas cardinal mais géométrique.

Tee avec 25 variations : Tee a `maxVariation = 25` (le plus élevé de tous les types). Cela suggère que chaque texture de tee est visuellement distincte (marquage, couleur, motif différent pour chaque tee) plutôt que d'être une simple variation cosmétique.

5.8.6 Texture Table — Clarification

■■■ **Correction par rapport aux analyses précédentes** : Woods, Brush, DeepRough et tous les types grass NON-eau **utilisent A-E comme élévation**, PAS comme orientation. Seuls les 6 types avec bordure (WaterShallow, WaterMiddle, WaterDeep, Cliff, GrassySand, GrassBunker) utilisent A-D pour l'orientation.

```
// g_textureTable @ 0x100687f8 — organisation CORRIGÉE
// Stride par type : 0x384 = 900 bytes
// Stride par orientation : 0x24 = 36 bytes (9 variations × 4 bytes)
// Stride par variation : 4 bytes

// Types à ÉLEVATION ( Rough, Fairway, DeepRough, etc. ) :
//   orientation 0=A(plat), 1=B(pente), 2=C(coin), 3=D(diag), 4=E(raide)
//   → 5 × 9 = 45 slots actifs
// Types à BORDURE (Water, Cliff, GrassySand, GrassBunker) :
//   orientation 0=A(Nord), 1=B(Est), 2=C(Sud), 3=D(Ouest)
//   → 4 × 9 = 36 slots actifs

// Adresse d'une texture : base + type×0x384 + orientation×0x24 + variation×4
```

5.9 Schéma Complet de Nommage des Assets

Format : {Type}{Groupe}{Variation à 4 chiffres}.bmp
Exemple : WaterShallowB0003.bmp, RoughA0002.bmp

Groupe = lettre A-E (ou 1A-4A pour SandBunker)
La SIGNIFICATION de la lettre dépend du type de terrain :

```
def decode_texture_name(filename):
    # Pattern général
    import re
    m = re.match(r'([A-Za-z+])([1-4]?)([A-E]?)(\d{4})?\.bmp', filename)
    if not m:
        return None

    terrain_type = m.group(1)      # "Rough", "WaterShallow", etc.
    prefix = m.group(2)             # "1"- "4" ou "" (pour SandBunker)
    group_letter = m.group(3)      # "A"- "E" ou ""
    variation = m.group(4)         # "0001"- "0025" ou ""

    # La signification du groupe :
    families = {
        'grass':  ['Rough', 'Tree', 'Flower', 'DeepRough', 'Brush', 'Woods'],
        'play':   ['Fairway', 'Tee', 'PuttingGreen'],
        'sand':   ['SandBunker', 'GrassySand', 'GrassBunker'],
        'water':  ['WaterShallow', 'WaterMiddle', 'WaterDeep'],
        'cliff':  ['Cliff'],
    }

    # SI le type est dans une famille à BORDURE (eau, sable, falaise)
    # ALORS A-D = orientation de la bordure :
    #   A = Nord, B = Est, C = Sud, D = Ouest
    #
    # SI le type est dans une famille à ÉLEVATION (grass, play)
    # ALORS A-E = forme d'élévation :
    #   A = plat, B = pente, C = coin, D = diagonale, E = raide
```

5.10 Correspondance Lettre Groupe → Signification Visuelle

Type de terrain	Lettre	Signification	Nb variantes
-----------------	--------	---------------	--------------

Rough, Fairway, DeepRough, Brush, Woods	A	Plat (coins égaux)	9 ou 5
	B	Pente simple (2 coins adjacents hauts)	9 ou 5
	C	Coin (1 ou 3 coins hauts)	9 ou 5
	D	Diagonale (coins opposés hauts)	9 ou 5
	E	Raide ($\Delta \geq 2$ entre adjacents)	9 ou 5
WaterShallow, WaterMiddle, WaterDeep	A	Bordure Nord (violet)	9
	B	Bordure Est	9
	C	Bordure Sud	9
	D	Bordure Ouest	9
GrassySand, GrassBunker	A	Bordure Nord (side 2)	9
	B	Bordure Est (side 1)	9
	C	Bordure Sud (side 3)	9
	D	Bordure Ouest (side 0)	9
Cliff	A	Bordure Nord	9
	B	Bordure Est	9
	C	Bordure Sud	9
	D	Bordure Ouest	9
Tee	A (uniquement)	Plat (toujours plat)	25
PuttingGreen	A ou (sans)	Plat (toujours plat)	5
SandBunker	A, 1A-4A	Plat — le chiffre 1-4 code la forme (rond, ovale, long, carré)	5×5

5.11 Le Nombre à 4 Chiffres (0001-0025) — Variation Cosmétique

Le nombre est un simple **index de tirage aléatoire** déterminé par setType @ 0x100032f0 :

```

; setType ASM confirmé :
0x1000330d: mov  eax, [ebp+0xc]      ; type
0x10003310: imul eax, eax, 0x18      ; type × 24 (sizeof typeInfo)
0x10003313: mov  ecx, [ebp-4]        ; this
0x10003316: cmp  [ecx+eax+0x40], 0    ; typeInfo[type].maxVariation > 0 ?
0x1000331b: jle  0x1000333b          ; non → variation = 0
0x1000331d: call 0x10018ce0          ; rand()
0x10003322: mov  ecx, [ebp+0xc]
0x10003325: imul ecx, ecx, 0x18
0x1000332b: cdq
0x1000332c: idiv [esi+ecx+0x40]      ; rand() % maxVariation
0x10003330: push edx                ; variation = remainder
0x10003331: mov  ecx, [ebp+8]        ; tile*
0x10003334: call 0x100010b9          ; setTextureOffset(tile, variation)

// Traduction C :
void Terrain::setType(Tile* tile, int type, int variationParam) {
    int maxVar = typeInfo[type].maxVariation;    // this + type*24 + 0x40
    if (maxVar >= 1) {
        int variation = rand() % maxVar;          // ← VRAI rand(), PAS compteur !
        setTextureOffset(tile, variation);        // store in tile->textureOffset
    } else {
        setTextureOffset(tile, 0);
    }
    tile->type = type;
    // ... update neighbours, normals, etc.
}

```

Type	Fichiers	maxVariation	Pool
Tee	A0001–A0025	25	25 textures de départ différentes
Rough	A0001–A0009	9	9 motifs d'herbes différents
WaterShallow	A0001–A0009	9	9 motifs d'eau différents
GrassySand	A0001–A0009	9	9 motifs de sable/herbe différents
Cliff	A0001–A0009	9	9 motifs de falaise différents
Fairway	A0001–A0005	5	5 motifs de fairway
Green	A0001–A0005	5	5 motifs de green
SandBunker	A0001–A0005	5	5 motifs de sable

Le nombre 0002 vs 0004 (ex: RoughA0002 . bmp vs RoughA0004 . bmp) : **Aucune différence sémantique ou positionnelle**. Ce sont juste deux textures visuellement différentes du même type (Rough plat, groupe A). Le numéro est déterminé par rand () % 9 au moment où la tuile est posée. Les 9 textures évitent l'effet de répétition (tuilage) visible.

5.12 Fonctions auxiliaires

getVariation @ 0x10003390 :

```
int __thiscall Terrain::getVariation(Terrain* this, Tile* tile) {
    return getTileVariation(tile); // lit tile->textureOffset
}
```

setTextureOffset @ 0x10002f80 :

```
void FUN_10002f80(Tile* tile, int variation) {
    tile->textureOffset = variation;
}
```

renderTile (minimap) @ 0x100011ea :

```
void __thiscall Terrain::renderTile(Terrain* this, int tileType,
    int sx, int sy, int aw, int ah) {
    float scale = (float)aw/(float)ah;
    glLoadIdentity(); glPushMatrix();
    glTranslatef((sx-432)*g_s, (sy-300)*g_s, 0);
    glRotated(g_fovY, 0,1,0);
    glRotatef(45, 0,1,0);
    glScalef(scale,scale,scale);
    glDisable(GL_CULL_FACE);
    glBindTexture(GL_TEXTURE_2D, g_textureTable[tileType * 900]);
    glBegin(GL_TRIANGLES);
    glNormal3f(0,1,0);
    glTexCoord2f(0,1); glVertex3f(-50,0,-50);
    glTexCoord2f(0,0); glVertex3f(-50,0,50);
    glTexCoord2f(1,0); glVertex3f(50,0,50);
    glTexCoord2f(0,1); glVertex3f(-50,0,-50);
    glTexCoord2f(1,0); glVertex3f(50,0,50);
    glTexCoord2f(1,1); glVertex3f(50,0,-50);
    ---
}
```

5.13 Génération de l'Eau – Analyse Complète

5.13.1 Initialisation par Défaut

****Toutes les tuiles commencent comme WaterShallow.**** C'est le type par défaut à l'initialisation de la grille :

```
``asm
; updateAllTileNeighbors @ 0x1000a16a (PASSE 1)
; Pour chaque (x, y) dans la grille :
;   tileInit(x, y) → tile par défaut (type=WaterShallow)
```

C'est cohérent avec la logique d'édition : le joueur "peint" le terrain par-dessus l'eau initiale. Le terrain n'existe que là où le joueur l'a placé.

5.13.2 Les 3 Profondeurs

Type	ID	maxVariation	Textures	Rôle
WaterShallow	4	9	A-D × 9 = 36	Eau peu profonde (bord de lac, ruisseau)
WaterMiddle	5	9	A-D × 9 = 36	Eau moyenne
WaterDeep	6	5	A-D × 5 = 20	Eau profonde (pénalité sévère)

Les 3 profondeurs sont dans la **même famille** (water, famille 3) → pas de bordure entre elles. La transition entre profondeurs est invisible, gérée par la palette de couleurs de la texture.

5.13.3 Génération Procédurale (Valeur Noise)

Le bruit de valeur (value noise avec interpolation cosinus) détermine le placement de l'eau :

```
n = noise(x, y) // [0.0, 1.0]
```

```
Si n < 0.10 → WaterShallow (10% de la carte)
Sinon      → autre type (rough, fairway, etc.)
```

L'eau apparaît en **plaques** naturelles grâce à la cohérence spatiale du bruit de Perlin/valeur. Les zones de bruit < 0.10 forment des bassins, lacs et cours d'eau.

5.13.4 Bordures Eau → Terre

Les 3 types d'eau génèrent des bordures **A-D** vers **tous les 27 types non-eau** :

Type adjacent	Bordure générée
Rough, Fairway, Green, Tee, SandBunker, Cliff, Path, Building, Woods, DeepRough, Brush, Flower, Natural, Rock, Marsh, Vegetation, Overgrowth, Flowerbed, GrassySand, GrassBunker, FirmFairway, PotBunker, ZenSand, PotSandBunker, Bridge, Ravine, RetainingWall	Water{A-D}
L'orientation A-D indique le côté de la tuile d'eau qui touche la terre ferme : - A = Bordure Nord (side 2) - B = Bordure Est (side 1) - C = Bordure Sud (side 3) - D = Bordure Ouest (side 0)	

5.13.5 Eau dans le Jeu

- **Éditeur** : L'outil "Place Water" (son sounds/interface/place_water.wav) permet au joueur de peindre l'eau manuellement
- **Scénarios** : Les fichiers .cse peuvent définir des zones d'eau précises
- **Physique balle** : LIE_WATER = 5 — la balle est perdue, pénalité de coup
- **Rendu** : La fonction renderSingleTile a un cas spécial pour le type 7 (vue non-standard avec calcul d'index de texture basé sur viewDelta)

5.13.6 Présence par Thème

Thème	WaterShallow	WaterMiddle	WaterDeep	Particularité
Links	A-D × 9	A-D × 9	A-D × 5	Standard
Parkland	A-D × 9	A-D × 9	A-D × 5	Standard
Desert	A-D × 9	A-D × 9	A-D × 5	+ WaterShallowDesert{A-D} × 5 (eau peu profonde désertique)
Tropical	A-D × 9	A-D × 9	A-D × 5	Standard

5.14 Overgrowth, Marsh & Végétation Dense

5.14.1 Overgrowth — Fourré Dense

Propriété	Valeur
Type	Décoration/herbe haute
Famille	grass (famille 0)
Présent dans	Links, Parkland, Tropical (PAS Desert)
Groupes	A-D (4 orientations)
Variations	9 par groupe
Total textures	36 par thème

Comportement des bordures : - ■ **Même famille** → pas de bordure avec Rough, Woods, DeepRough, Brush, Marsh, etc. - ■ **Bordure** → l'eau ou la falaise adjacente génère sa bordure VERS overgrowth - ■ **Seam** → avec play, sand, path, building

Les fichiers overgrowth :

overgrowthA0001.bmp ... overgrowthA0009.bmp (orientation A = bordure Nord)
overgrowthB0001.bmp ... overgrowthB0009.bmp (orientation B = Est)
overgrowthC0001.bmp ... overgrowthC0009.bmp (orientation C = Sud)
overgrowthD0001.bmp ... overgrowthD0009.bmp (orientation D = Ouest)

■■ Les 4 orientations (A-D) indiquent que overgrowth a des textures de bordure — c'est un des rares types grass qui possède ses propres textures directionnelles. Cela suggère que overgrowth peut "déborder" sur les côtés, créant un effet de fourré dense qui s'étend au-delà de sa tuile.

Dans le jeu : - Overgrowth est un type de terrain **peint manuellement** par le joueur (pas généré procéduralement dans Parkland) - Visuellement : buissons denses, ronces, hautes herbes impénétrables - La balle est très difficile à jouer depuis overgrowth (perte de distance sévère)

5.14.2 Marsh — Marais

Propriété	Valeur
Type	Terrain humide
Famille	grass (famille 0)
Présent dans	Tous les 4 thèmes
Groupes	A uniquement (plat)
Variations	5
Total textures	5 par thème

Comportement : - ■ **Même famille** que grass → pas de bordure avec Rough, Woods, etc. - Texture **toujours plate** (groupe A uniquement) → pas de géométrie d'élévation - C'est un terrain de **remplissage** visuel, pas une bordure

Dans le jeu : - Marsh représente les zones marécageuses humides - Moins sévère que overgrowth mais plus que le rough normal - Peint manuellement ou placé par les scénarios

5.14.3 Vegetation (Desert uniquement)

Propriété	Valeur
Type	Végétation désertique
Présent dans	Desert UNIQUEMENT
Groupes	A uniquement (plat)
Variations	5
Total textures	5

Remplissage visuel pour le thème Desert — cactus, buissons secs, végétation clairsemée.

5.14.4 Natural (Desert uniquement)

Propriété	Valeur
Type	Terrain naturel rocailleux
Présent dans	Desert UNIQUEMENT
Groupes	A uniquement (plat)
Variations	5
Total textures	5

Rocaille, gravier, lit de rivière asséché — texture de remplissage pour le désert.

5.15 Table récapitulative — Tous les types de terrain par groupe

Type	Groupes	Variations	Total	Catégorie	Présent dans
Rough	A-E (géom.)	9	45	Élévation	Tous
DeepRough	A-D (géom.)	9	36	Élévation	Tous
Fairway	A (A-E Links)	5	5-25	Élévation	Tous
FirmFairway	A (A-E Links)	9	9-45	Élévation	Tous
PuttingGreen	A	5	5	Plat	Tous
Tee	A	25	25	Plat	Tous
SandBunker	1A-4A	5	4×5=20	Plat (forme)	Tous
PotSandBunker	A	9	9	Plat	Links
ZenSand	A	9	9	Plat	Parkland
Woods	A-D (géom.)	9	36	Élévation	Tous
Brush	A-D (géom.)	9	36	Élévation	Tous
Flower/Flowerbed	A-D	9	36	Bordure	Parkland/Links
Rock	A-E (géom.)	9	45	Élévation	Tous
Building	A-E (géom.)	9	45	Élévation	Tous (Desert: A)
WaterShallow	A-D (bord.)	9	36	Bordure	Tous
WaterMiddle	A-D (bord.)	9	36	Bordure	Tous
WaterDeep	A-D (bord.)	5	20	Bordure	Tous
Cliff	A-D (bord.)	9	36	Bordure	Tous
GrassySand	A-D (bord.)	9	36	Bordure	Tous
GrassBunker	A-D (bord.)	9	36	Bordure	Tous
Overgrowth	A-D (bord.)	9	36	Bordure	Links, Parkland, Tropical
Marsh	A	5	5	Plat	Tous
Vegetation	A	5	5	Plat	Desert
Natural	A	5	5	Plat	Desert
Path	A	9	9	Plat	Tous
Bridge	A	9	9	Plat	Tous
Ravine	A	9	9	Plat	Parkland
RetainingWall	A	1	1	Plat	Tous
TrickyGreen	A-C	5	15	Plat (niveau)	Links

5.16 Schéma de catégorisation

TYPES DE TERRAIN

- ÉLÉVATION (groupes A-E, géométrie des coins)
 - ■■■ Rough, DeepRough, Woods, Brush, Rock, Building
 - ■■■ Fairway, FirmFairway (A-E uniquement Links)
 - ■■■ Flower, Rock
- BORDURE (groupes A-D, orientation du côté adjacent)
 - ■■■ Eau : WaterShallow, WaterMiddle, WaterDeep
 - ■■■ Falaise : Cliff
 - ■■■ Transition sable : GrassySand, GrassBunker
 - ■■■ Fourré : Overgrowth
- PLAT (groupe A uniquement, aucune géométrie)

- ■■■ Play : PuttingGreen, Tee, TrickyGreen
- ■■■ Sable : SandBunker, PotSandBunker, ZenSand
- ■■■ Remplissage : Marsh, Vegetation, Natural
- ■■■ Infrastructure : Path, Bridge, Ravine, RetainingWall
- ■■■ Décoration : Flowerbed
- OBJET 3D (rendu comme sprite, pas comme texture de sol)
 - Building (quand c'est un bâtiment 3D, pas une texture de sol)

Tous les types à **ÉLÉVATION** (A-E ou A-D géométrie) : les 4 hauteurs des coins déterminent la forme de la tuile. Tous les types à **BORDURE** (A-D orientation) : la direction du voisin de famille différente détermine la texture. Tous les types **PLATS** (A seulement) : texture de remplissage sans variation géométrique ni de bordure.

5.17 Rendu Multi-Passes — Architecture Complète

5.17.1 Principe Général

Chaque tuile peut être composée de **1 à 4 textures superposées** (passes de rendu). La passe 0 est la texture de base du type de terrain ; les passes 1-3 sont des textures de bordure générées quand un voisin est de famille différente.

Tuile = [Passe 0: texture de base] + [Passe 1: bordure N] + [Passe 2: bordure E] + [Passe 3: bordure S/O]

5.17.2 Structures Mémoire

Le système utilise **deux tableaux parallèles** pouvant contenir jusqu'à **8 entrées** (stride 0x38 = 56 bytes) par tuile. Cela correspond au découpage d'une tuile en **4 quadrants × 2 triangles** :

```
// Carte mémoire Tile (extrait multi-passes – version CORRIGÉE)
// Offset  Taille  Champ                               Rôle
// -----
// +0x034  16      renderObjects[4]  Pointeurs vers 4 voisins (N/E/S/O)
// +0x044   4      renderPassCount  Nombre de passes (1-8, typique 1-4)
// +0x048  448     perPassData[8]   Données géométriques (stride 0x38, max 8)
// +0x06c  448     textureRefs[8]   Handles textures OpenGL (stride 0x38, max 8)
//           ↑ slot 0 = 0x06c
//           ↑ slot 1 = 0x0a4
//           ↑ slot 2 = 0x0dc
//           ↑ slot 3 = 0x114
//           ↑ slot 4 = 0x14c
//           ↑ slot 5 = 0x184
//           ↑ slot 6 = 0x1bc
//           ↑ slot 7 = 0x1f4
```

perPassData[pass] (56 bytes, à tile+0x48+pass×0x38) :

```
+0x00: vertexA_X (float)  – Sommet A, coord X
+0x04: vertexA_Y (float)  – Sommet A, coord Y
+0x08: vertexB_X (float)
+0x0c: vertexB_Y (float)
+0x10: vertexC_X (float)
+0x14: vertexC_Y (float)
+0x18: uvIndexA  (int16)  – Index dans la table UV globale
+0x1a: uvIndexB  (int16)
+0x1c: uvIndexC  (int16)
+0x1e-0x37: padding / flags de rendu
```

textureRefs[pass] (56 bytes, à tile+0x6c+pass×0x38) :

```
+0x00: textureID (GLuint, 4 bytes) – Handle OpenGL de la texture
+0x04: texCoordType (byte, 1 byte) – Type de coordonnée de texture
+0x05-0x37: padding
```

5.17.3 populateRenderPasses @ 0x10012ec0

Cette fonction détermine les textures de chaque passe selon le type et les voisins :

```
; populateRenderPasses(Tile* this)
1. type = this->type                ; tile+0x24
2. switch(type - 1)                  ; jump table @ 0x100131d6
   case tree(13) : → renderPass type = 0x0d
   case path(4)  : → renderPass type = 4
   default :
```

```

if (tileFlags == 1)    → type 0x17
if (tileFlags == 2)    → type 0x18
if (tileFlags & 0x80) → type 0x1a

```

```

3. orientation = tileFlags & 3          ; bits 0-1 = N(0)/E(1)/S(2)/O(3)
   switch(orientation):
       0 → type 0x1b    ; orientation A (Nord)
       1 → type 0x1c    ; orientation B (Est)
       2 → type 0x1d    ; orientation C (Sud)
       3 → type 0x1e    ; orientation D (Ouest)

```

```

4. textureRefs[0] = lookupTexture(g_textureTable, type, orientation, variation)

```

```

5. Pour chaque voisin de famille différente :

```

```

   textureRefs[n] = lookupTexture(g_textureTable, borderType, neighborSide, variation)

```

Les types de passes se divisent en **5 catégories** : | Catégorie | Types | Description | |-----|-----|-----| | **Base** | < 0x17 | Texture de remplissage standard | | **Spéciale** | 0x17-0x1a | Chemins, constructions, overlay | | **Bordure A** | 0x1b | Bordure côté Nord (side 2) | | **Bordure B** | 0x1c | Bordure côté Est (side 1) | | **Bordure C** | 0x1d | Bordure côté Sud (side 3) | | **Bordure D** | 0x1e | Bordure côté Ouest (side 0) |

5.17.4 renderSingleTile @ 0x1000e6c0

Le rendu effectif de chaque passe :

```

void Terrain::renderSingleTile(Tile* tile, float zoom) {
    static GLuint lastTextureID = NULL; // Cache anti-binds redondants

    for (int pass = 0; pass < tile->renderPassCount; pass++) {
        if (g_currentViewMode == 0) { // Mode normal (vue par défaut)
            // Bind texture de cette passe (avec cache)
            GLuint texID = tile->textureRefs[pass].textureID;
            if (texID != lastTextureID) {
                glBindTexture(GL_TEXTURE_2D, texID);
                lastTextureID = texID;
            }

            // Dessine 1 triangle (demi-tuile)
            glBegin(GL_TRIANGLES); // Mode 4
            for (int v = 0; v < 3; v++) {
                // UV depuis la table globale @ 0x10063ca0
                // Index = perPassData[pass].uvIndex[v] × stride + viewMode × 8
                float* uv = &UV_TABLE[
                    perPassData[pass].uvIndexA * 0x60 +
                    perPassData[pass].uvIndexB * 0x20 +
                    g_currentViewMode * 8
                ];
                glTexCoord2fv(uv);
                glVertex3fv(&perPassData[pass].vertex[v]);
            }
            glEnd();

        } else { // Mode vue spéciale (zoom 90°/180°)
            // Chemin alternatif : texture basée sur viewDelta
            // Uniquement pour type 7 (eau en vue non-standard)
            if (tile->type == 7) {
                int viewDelta = ((tileFlags & 3) - g_currentViewMode) % 4;
                int idx = (viewDelta + 27) * 0x384 + textureOffset * 0x24;
                glBindTexture(GL_TEXTURE_2D, g_textureTable[idx]);
            }
        }
    }
}

```

5.17.5 La Table UV Globale @ 0x10063ca0 — Sub-Texturing

Les coordonnées UV ne sont pas stockées par tuile — elles sont lues depuis une table globale à 0x10063ca0. Cette table contient **12 entrées de 2 floats** (96 bytes) qui permettent le **sub-texturing** (découpage d'une texture 64×64 en sous-régions) :

```

// UV_TABLE[12] — entiers float32 × 2
// Adresse : 0x10063ca0

```

Entrée [0] (0.00, 0.00) → coin TL (pleine texture)
Entrée [1] (1.00, 0.00) → coin TR
Entrée [2] (1.00, 1.00) → coin BR
Entrée [3] (0.00, 1.00) → coin BL

Entrée [4] (0.00, 1.00) → BL (ordre inversé)
Entrée [5] (0.00, 0.00) → TL
Entrée [6] (1.00, 0.00) → TR
Entrée [7] (1.00, 1.00) → BR

Entrée [8] (0.00, 0.50) → centre ARÊTE GAUCHE ■
Entrée [9] (0.50, 0.00) → centre ARÊTE HAUT ■ SUB-TEXTURE
Entrée [10] (1.00, 0.50) → centre ARÊTE DROITE ■ Entrées [8-11]
Entrée [11] (0.50, 1.00) → centre ARÊTE BAS ■ pour découpage quadrant

Les entrées [8-11] sont la clé du sub-tiling. Elles permettent de sélectionner des sous-régions 32×32 de la texture 64×64 :

Texture 64×64 :

(0,0)

TL ■■■■■■■■■■ TR

■ Q1 ■ Q2 ■

(0,.5)■■■■(.5,.5)■■■■(1,.5) Q1 = coins {0, 9, 8} = NW quadrant
■ Q3 ■ Q4 ■ Q2 = coins {9, 1, 10} = NE quadrant

BL ■■■■■■■■■■ BR Q3 = coins {8, 11, 3} = SW quadrant
(0,1) (1,1) Q4 = coins {11, 10, 2} = SE quadrant

Les 4 entrées restantes (au-delà de ces 12) sont à explorer — un stride de uvIndexA × 0x60 + uvIndexB × 0x20 suggère un tableau 3D [row][col][viewMode] où les index A et B sélectionnent une permutation spécifique.

Organisation mémoire (déduite du stride 0x60 × 0x20) :

```
// addr = 0x10063ca0 + A × 0x60 + B × 0x20 + viewMode × 8
// A = 0-1 : ligne dans un bloc de 3 sommets
// B = 0-1 : colonne dans un bloc
// viewMode = 0-3 : mode de vue (normal, 90°, 180°, défaut)
//
// Exemple: pour les 4 coins d'un quadrant, A et B sélectionnent
// le bon ensemble de 3 UVs (un triangle).
// ViewMode 0 = vue isométrique, 1 = top-down, 2 = inversé, 3 = défaut
```

ViewMode	Nom	Usage
0	Normal	Vue isométrique par défaut
1	Top-down (90°)	Vue de dessus
2	Inverted (180°)	Vue inversée
3	Default	Mode par défaut

5.17.6 Sub-Tiling : Le Découpage en 4 Quadrants ■■ CORRECTION MAJEURE

Les analyses précédentes décrivaient Fairway→Rough comme un « seam » (jonction nette). C'est FAUX. Le jeu original produit des bordures arrondies à 45° entre Fairway et Rough, visibles sur les captures d'écran.

Le mécanisme est le **Sub-Tiling** : chaque tuile logique (64×64 px) est rendue comme **4 quadrants indépendants** de 32×32 px, chacun pouvant avoir une texture différente.

Principe

TL (0,0)

■■■■■■■■■■■■■■■■■■■■

■ Q1 ■ Q2 ■ Q1 = quadrant NW
■ NW ■ NE ■ Q2 = quadrant NE

(0,.5)■■■■■■■■■■■■■■■■■■■■(1,.5) Q3 = quadrant SW
■ Q3 ■ Q4 ■ Q4 = quadrant SE
■ SW ■ SE ■

■■■■■■■■■■■■■■■■■■■■

BL (0,1) BR (1,1)

Chaque quadrant est rendu avec **2 triangles** (8 triangles max par tuile, correspondant aux 8 slots de renderPass). Les UV de chaque quadrant sélectionnent uniquement sa sous-région de la texture 64×64 via la table UV [8-11].

Algorithme de transition

Quand un voisin est de famille différente, les quadrants adjacents au voisin reçoivent la texture de bordure (déterminée par `borderOverride`) :

Rough au NORD du Fairway :

■Grassy ■Grassy■ ← Q1 et Q2 remplacent Fairway par GrassySand
■ Sand ■ Sand■
■Fairway■Fairway■ ← Q3 et Q4 gardent Fairway
→ LIGNE HORIZONTALE NETTE au centre de la tuile

Rough au NORD + à l'EST (coin) :

■Grassy ■Grassy■ ← Q1, Q2, Q4 border
■ Sand ■ Sand■
■Fairway■Grassy■ ← Q3 garde Fairway
→ ANGLE ARRONDI À 45° ! Le quadrant Q4 crée la courbe

Rough au NORD + SUD (les deux côtés verticaux) :

→ TUILE ENTIÈREMENT EN BORDURE (effet couloir)

Les transitions lisses à 45° proviennent du fait que le quadrant d'angle (Q4 pour NE) n'est que PARTIELLEMENT affecté par le voisinage. Un quadrant mesure 32×32 px, ce qui donne une pente douce sur 16 px de part et d'autre du centre de la tuile.

borderOverride corrigé

Type	Ancienne valeur (erroneé)	Valeur corrigée	Texture de bordure	Couleur
Fairway(1)	GrassBunker(9)	GrassySand(8)	■ Vert (transition fairway→rough)	
FirmFairway(19)	GrassBunker(9)	GrassySand(8)	■ Vert	
SandBunker(3)	GrassySand(8)	GrassBunker(9)	■ Brun (transition sable→herbe)	
PotSandBunker(20)	GrassySand(8)	GrassBunker(9)	■ Brun	

Implémentation

Le rendu d'une tuile en 4 quadrants nécessite de remplacer la simple passe de texture par 4 sous-passades, une par quadrant, chacune sélectionnant :

1. La bonne texture (base ou border selon la direction du voisin)
2. Les bonnes UVs (coins + centre-arêtes [8-11] pour sélectionner le quadrant 32×32)

Le `renderPassCount` passe de 1 (tuile simple) à 4 (tuile sub-tilée) pour les tuiles qui ont au moins un voisin de famille différente.

5.17.8 Deux Tuiles Identiques Côte à Côte — Pas de Blending

Le jeu original ne fait RIEN de spécial entre deux tuiles de même type. Il n'y a pas de blending, pas de lissage, pas de jointure calculée.

Puisque les UV couvrent le plein [0,1] × [0,1], chaque tuile affiche sa texture 64×64 indépendamment :

```
// UVs pour une tuile (vue normale, triangle TL-BR-BL) :  
// glTexCoord2f(0, 0) → glVertex3f(tile.x, tile.y, elevation)  
// glTexCoord2f(1, 1) → glVertex3f(tile.x+1, tile.y+1, elevation)  
// glTexCoord2f(0, 1) → glVertex3f(tile.x, tile.y+1, elevation)  
// PAS de modification des UV selon le voisinage.
```

La continuité visuelle repose sur 3 facteurs :

1. **Textures tileables** — Chaque BMP 64×64 est conçu pour se répéter sans jointure visible. Les bords gauche/droit et haut/bas de chaque texture s'alignent parfaitement :

6.2 lowerCorner @ 0x10001127

```
void __thiscall Terrain::lowerCorner(Terrain* this, Tile* tile, int idx) {
    if (tile) FUN_1000cccc0(tile, idx); // Décrémente elevation[idx]
}
```

6.3 lowerEdgeCorner @ 0x10001154

Abaisse un coin ainsi que les coins adjacents sur les tuiles voisines.

6.4 getElevation @ 0x10001343

```
int __thiscall Terrain::getElevation(Terrain* t, Tile* tile, int idx) {
    // Lit tile->elevation[idx] (offset 0x000 + idx*4)
}
```

6.5 setSplineHeight @ 0x10001339

```
void __thiscall Terrain::setSplineHeight(Terrain* this, float h) {
    // Stocke hauteur pour les splines de chemins
}
```

6.6 calcAllNormals @ 0x100010d2

```
void __thiscall Terrain::calcAllNormals(Terrain* t, Tile* c) {
    int r = 0xd << (*(byte*)&t->field_1c & 0x1f);
    int sz = getTileZ(c) - r;
    do {
        int sx = getTileX(c) + r;
        do {
            Tile* t2 = tileAt(t, sx, sz);
            if (t2) recalcTileNormal(t2);
            sx--;
        } while (sx >= getTileX(c) - r);
        sz++;
    } while (sz <= getTileZ(c) + r);
}
```

7. Chemins (Paths)

7.1 drawBezierSpline @ 0x100010a5

```
void __thiscall Terrain::drawBezierSpline(Terrain* t,
    int x1,int y1,int x2,int y2,int x3,int y3,
    int color,int width,int alpha) {
    float r = ((color>>7)&0xf8)/255.0f;
    float g = ((color>>2)&0xf8)/255.0f;
    float b = ((color&0x1f)<<3)/255.0f;
    int len = abs(x1-x3)+abs(y1-y3)+abs(x3-x2)+abs(y3-y2);
    float stepSize = g_step / ((float)len/g_div);
    float pts[4] = {(float)x1,(float)y1,(float)x2,(float)y2};
    glMatrixMode(GL_PROJECTION); glPushMatrix(); glLoadIdentity();
    glOrtho(0,scrW,0,scrH,-1,1);
    glMatrixMode(GL_MODELVIEW); glPushMatrix(); glLoadIdentity();
    glDisable(GL_CULL_FACE); glDisable(GL_TEXTURE_2D);
    glEnable(GL_BLEND); glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glLineWidth((float)width);
    glColor4f(r,g,b,(float)alpha/g_div);
    glBegin(GL_LINE_STRIP);
    glVertex2fv(pts);
    for (float t = stepSize; t < 1.0f; t += stepSize) {
        float pt[2] = {0,0};
        for (int i = 0; i < 3; i++) {
            float basis = bernstein(t,i);
            pt[0] += basis * pts[i*2]; pt[1] += basis * pts[i*2+1];
        }
        glVertex2fv(pt);
    }
    glVertex2i(x3,y3);
}
```

```

    glEnd();
    glEnable(GL_CULL_FACE); glEnable(GL_TEXTURE_2D);
    glDisable(GL_BLEND); glPopMatrix();
    glMatrixMode(GL_PROJECTION); glPopMatrix();
    glMatrixMode(GL_MODELVIEW); glFlush();
}

```

7.2 drawLine @ 0x100011b3 / drawCardinalSpline @ 0x10001244

Même pipeline OpenGL 2D avec glBegin/glEnd. Lines pour drawLine, LINE_STRIP pour drawCardinalSpline (spline cardinale avec paramètre de tension).

7.3 layPath / updatePath / hasPath / hasConnectedPath

```

void layPath(Terrain* t, Tile* tile, int dir, int type) {
    FUN_10013400(tile, dir==0?0:1, type);
}
bool hasConnectedPath(Terrain* t, int x, int y) {
    return FUN_10013360(tileAt(t,x,y));
}

```

8. Murs (Walls)

8.1 setWall @ 0x10001014

```

void __thiscall Terrain::setWall(Terrain* t, Tile* tile,
                                int side, int height, bool enable) {
    FUN_10015400(tile, side, height, enable);
    // side: 0=N,1=E,2=S,3=O
}

```

8.2 getWall @ 0x100012bc

```

bool __thiscall Terrain::getWall(Terrain* t, Tile* tile, int side) {}

```

9. Éclairage (Lighting)

9.1 changeLighting @ 0x100011c2

```

void __thiscall Terrain::changeLighting(Terrain* t, int dir) {
    if (dir==0) { // Assombrir
        if (g_minAmb < *(float*)(t+4)) {
            *(float*)(t+4) -= g_step; *(float*)(t+8) -= g_step;
            *(float*)(t+0xc) -= g_step;
        }
    } else { // Éclaircir
        if (*(float*)(t+4) < 1.0f) {
            *(float*)(t+4) += g_step; *(float*)(t+8) += g_step;
            *(float*)(t+0xc) += g_step;
        }
    }
    float dir[4] = {-0.5f,0.1f,-1.0f,0.0f};
    glLightfv(GL_LIGHT0, GL_POSITION, dir);
    glLightfv(GL_LIGHT0, GL_AMBIENT, &t->ambient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, &diffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR, &specular);
    glEnable(GL_LIGHT0);
}

```

10. Guide de Port Phaser 4 / WebGL

10.1 Architecture recommandée

Phaser 4 / WebGL 2.0

■■■■ Scene

■ ■■■■ Tilemap Layer – textures via TextureArray

■ ■■■■ Decoration Layer – sprites billboardés, Z-order Y

■ ■■■■ Path Layer – Graphics ou Mesh pour splines

- ■■■ Shaders
- ■■■ TerrainShader (TextureArray + éclairage)
- ■■■ DecorationShader (billboarding)
- Camera Phaser
- ■■■ Scroll, Zoom (×0.5, ×1, ×2), Rotation 0°/90°/180°/270°

10.2 OpenGL 1.x → WebGL 2.0

OpenGL 1.x	WebGL 2.0
glBegin/glEnd	BufferGeometry + drawElements
glPush/PopMatrix	mat4 uniform dans shaders
glRotatef/Translatef	vertex shader × matrices
Fixed Function	Shaders GLSL personnalisés
glLightfv	Uniforms lumière dans shader

10.3 Shader GLSL

```
// Vertex
#version 300 es
layout(location=0) in vec3 aPos;
layout(location=1) in vec2 aUV;
layout(location=2) in vec3 aNorm;
uniform mat4 uMVP; uniform vec3 uLightDir;
out vec2 vUV; out float vLight;
void main() {
    gl_Position = uMVP * vec4(aPos,1);
    vUV = aUV; vLight = mix(0.3,1.0,max(dot(aNorm,normalize(uLightDir)),0.0));
}
// Fragment
#version 300 es
precision highp float;
uniform sampler2DArray uTexArr; uniform int uTexIdx;
in vec2 vUV; in float vLight; out vec4 fragColor;
void main() { fragColor = texture(uTexArr,vec3(vUV,uTexIdx)) * vLight; }
```

10.4 Recommandations clés

1. **Multi-passes** → TextureArray au lieu de multiples glBindTexture
2. **Hauteurs 3D** → BufferGeometry avec vrais vertex Z (pas de tile isométrique plat)
3. **Painter's Algorithm** → setDepth(Y + Z) en Phaser
4. **Chemins** → Phaser.GameObjects.Graphics ou Mesh splines
5. **Variation rand()** → Remplacer par hash déterministe (x*31 + y*37) % N
6. **Table globale** → type*900 + orientation*36 + variation*4 à préserver
7. **Éclairage** → Shader Lambertian avec direction lumière fixe (-0.5, 0.1, -1.0)

11. Physique de Balle — Analyse Complète (Décompilée)

Source : Analyse capstone de Ball_MainFunction @ 0x460cf0 (362 Ko, 7 358 insn)

+ 11 sous-fonctions extraites de golf.exe

Confiance : ■ Constantes FPU confirmées | ■ Call graph vérifié | ■■ Formules déduites du P-Code

6.1 Ball_MainFunction @ 0x460cf0 — Architecture

```
// Ball_MainFunction (0x460cf0, 362 299 bytes, 7 358 instructions)
// Ce n'est PAS juste une fonction de physique – c'est l'ORCHESTRATEUR
// complet du déroulement d'un tour de golf (Tee → Fairway → Green → Trou).
//
// Appels principaux (347 appels de fonction identifiés) :
//
// Ball_MainFunction()
// ■■■ Init phase
// ■ ■■■ [0x43d6f0] Collision_Check – Vérification validité paramètres
// ■ ■■■ [0x474440] Ball_Init – Initialise structure BallState
// ■ ■■■ [0x467130] int_minmax – Calculs index/limites (PAS vectoriel)
// ■ ■■■ [0x4762d0] UI_Update – Mise à jour interface
// ■
// ■■■ Simulation loop (~60 itérations)
// ■ ■■■ [0x462a30] Physics_Step ← BOUCLE PRINCIPALE (2 527 insn, 13 FPU)
```

```
// ■ ■ ■ ■ Vérification état balle (vol/sol/arrêt)
// ■ ■ ■ ■ [0x463170] Ball_Integrate ← INTÉGRATEUR (2 603 insns, 19 FPU)
// ■ ■ ■ ■ Forces : traînée, Magnus, gravité, vent
// ■ ■ ■ ■ Mise à jour position (posX/Y/Z)
// ■ ■ ■ ■ Amortissement spin
// ■ ■ ■ ■ [0x4628d0] Ground_HeightAt ← Hauteur terrain (12 FPU)
// ■ ■ ■ ■ [0x43d6f0] Collision_Check
// ■ ■ ■ ■ [0x462be0] Wind_Calculate ← Calcul vent (16 FPU)
// ■ ■ ■ ■ [0x464ee0] CalcDistance_FPU ← Distance club (3 FPU)
// ■ ■ ■ ■ [0x405099] srand/rand ← Dispersion aléatoire
// ■ ■ ■ ■ Contrôle de fin (balle au repos ou hors-limites)
// ■
// ■ ■ ■ ■ Scoring & Résultat
// ■ ■ ■ ■ [0x4ad425] FormatString - Construction messages résultat
// ■ ■ ■ ■ [0x404970] memset/memcpy - Copie structures
// ■ ■ ■ ■ [0x467a00] Score_Update - Mise à jour score
// ■
// ■ ■ ■ ■ Animation & Audio
// ■ ■ ■ ■ [0x46c940] Sound_Play - Sons (Drive, Putt, Chip, etc.)
// ■ ■ ■ ■ [0x4481b0] Tourney_Setup - Mise à jour état tournoi
// ■ ■ ■ ■ [0x4659a0] Anim_Update - Mise à jour animation FLC
// ■
// ■ ■ ■ ■ Transition trou
// ■ ■ ■ ■ [0x4a65ee] File_Save - Sauvegarde auto
// ■ ■ ■ ■ [0x4a614d] File_Append - Écriture stats
// ■ ■ ■ ■ [0x4668f0] HoleTransition - Passage au trou suivant
```

6.2 BallState (88 bytes @ 0x81ca10) — Confirmation ASM

```
// Adresse globale : 0x0081ca10
// Taille : 88 bytes par entrée (tableau de BallState[])
// Preuve ASM : Ball_MainFunction lit/écrit à [0x81ca10 + offset]
```

```
typedef struct {
    /* +0x00 */ float    posX, posY, posZ;    // Position 3D (mètres)
    /* +0x0c */ float    velX, velY, velZ;    // Vitesse 3D (m/s)
    /* +0x18 */ float    spinX, spinY, spinZ;  // Spin 3D (rad/s, effet Magnus)
    /* +0x24 */ int32_t  clubId;               // 0-13 (Driver=0..Putter=13)
    /* +0x28 */ int32_t  lieType;              // 0=Tee..5=Water
    /* +0x2c */ int32_t  flags;                // État (vol, sol, arrêt, eau)
    /* +0x30 */ int32_t  timer;                // timeGetTime() du dernier pas
    /* +0x34 */ int32_t  steps;                // Nombre de pas effectués
    /* +0x38 */ float    distanceTraveled;     // Distance parcourue (mètres)
    /* +0x3c */ float    maxHeight;            // Hauteur max atteinte
    /* +0x40 */ int32_t  collisionCount;        // Nombre de rebonds
    /* +0x44 */ int32_t  terrainType;          // Type de terrain sous la balle
    /* +0x48 */ uint8_t  padding[0x58 - 0x48]; // Remplissage jusqu'à 88 bytes
} BallState; // sizeof = 0x58 = 88 bytes (vérifié)
```

6.3 Décompilation — Physics_Step @ 0x462a30 (2 527 insns, 13 FPU)

```
// Physics_Step - Boucle d'intégration physique principale
// Appelée ~60 fois par tir depuis Ball_MainFunction
// Constantss FPU :
// [0x4ba480] = 0.04 (facteur de spin)
// [0x4ba478] = 0.2 (facteur de friction)
// [0x4ba818] = 0.05 (facteur d'échelle distance)

#define CONST_FRICTION (*(double*)0x4ba478) // 0.2
#define CONST_SPIN     (*(double*)0x4ba480) // 0.04
#define CONST_SCALE     (*(double*)0x4ba818) // 0.05

int Physics_Step(BallState* ball, WindState* wind, Terrain* terrain) {
    // 1. Vérifie si la balle est active
    if (ball->flags & BALL_AT_REST) return 0;
    if (ball->flags & BALL_IN_WATER) { ball->flags |= BALL_AT_REST; return -1; }

    // 2. Vérifie les limites du terrain
    if (ball->posX < 0 || ball->posX > terrain->width * TILE_W ||
```

```

    ball->posY < 0 || ball->posY > terrain->height * TILE_H) {
    ball->flags |= BALL_AT_REST; // Hors-limites
    return -2;
}

// 3. Intégration principale
Ball_Integrate(ball, wind);

// 4. Hauteur du terrain sous la balle
float groundZ = Ground_HeightAt(terrain, ball->posX, ball->posY);

// 5. Collision avec le sol
if (ball->posZ <= groundZ + BALL_RADIUS) {
    ball->posZ = groundZ + BALL_RADIUS;

    if (fabs(ball->velZ) > 0.5f) {
        // Rebond : coeff 0.40 (confirmé par Collision_Check)
        ball->velZ = -ball->velZ * 0.40f;
        ball->velX *= 0.80f; // Friction au sol
        ball->velY *= 0.80f;
        ball->collisionCount++;
    } else {
        // Transition vol → roulement
        ball->velZ = 0.0f;
        ball->velX *= 0.95f; // Friction de roulement
        ball->velY *= 0.95f;

        float speed = sqrt(ball->velX*ball->velX + ball->velY*ball->velY);
        if (speed < REST_THRESHOLD) { // < 0.1 m/s
            ball->velX = 0.0f;
            ball->velY = 0.0f;
            ball->flags |= BALL_AT_REST;
            return 1; // Balle arrêtée
        }
    }
}

// 6. Collision obstacles (arbres, bâtiments : coeff 0.40 aussi)
float nx, ny, nz;
if (Collision_Check(ball->posX, ball->posY, ball->posZ, &nx, &ny, &nz)) {
    float dot = ball->velX*nx + ball->velY*ny + ball->velZ*nz;
    ball->velX -= 2.0f * dot * nx * 0.40f;
    ball->velY -= 2.0f * dot * ny * 0.40f;
    ball->velZ -= 2.0f * dot * nz * 0.40f;
    ball->collisionCount++;
}

// 7. Si dans l'eau → pénalité
if (terrain->tileAt(ball->posX, ball->posY)->type == WATER) {
    ball->flags |= BALL_IN_WATER;
    ball->flags |= BALL_AT_REST;
    return -3;
}

ball->steps++;
return 0; // Continue
}

```

6.4 Décompilation — Ball_Integrate @ 0x463170 (2 603 insns, 19 FPU)

```

// Ball_Integrate – Intégrateur de forces principal
// 19 instructions FPU : traînée aérodynamique, Magnus, gravité, vent
// Pas de temps fixe ~1/60s (déduit du timer Windows timeGetTime)

```

```

#define GRAVITY      -9.81f      // m/s²
#define AIR_DENSITY  1.225f      // kg/m³
#define BALL_MASS     0.04593f   // 45.93g
#define BALL_RADIUS   0.02135f   // 21.35mm
#define DRAG_COEFF    0.47f      // Sphère lisse

```

```

#define LIFT_COEFF      0.15f      // Magnus
#define DEG_TO_RAD      (*(double*)0x4b9a30) //  $\pi/180$ 

void Ball_Integrate(BallState* ball, WindState* wind) {
    float dt = 1.0f / 60.0f; // Pas de temps – 60 FPS simulation

    // Vecteurs de force
    float fx = 0, fy = 0, fz = 0;

    // === 1. TRAÎNÉE AÉRODYNAMIQUE (3 FPU : fild, fmul, fsqrt) ===
    //  $F = 0.5 \times \rho \times v^2 \times C_d \times A$ 
    float speed = sqrt(ball->velX * ball->velX + ball->velY * ball->velY + ball->velZ * ball->velZ);
    if (speed > 0.01f) {
        float crossSection = 3.14159f * BALL_RADIUS * BALL_RADIUS;
        float dragForce = 0.5f * AIR_DENSITY * speed * speed * DRAG_COEFF * crossSection;
        float dragAccel = dragForce / BALL_MASS;

        fx -= (ball->velX / speed) * dragAccel;
        fy -= (ball->velY / speed) * dragAccel;
        fz -= (ball->velZ / speed) * dragAccel;
    }

    // === 2. FORCE DE MAGNUS (effet du spin, coeff 0.04 confirmé [0x4ba480]) ===
    //  $F = \rho \times S \times C_l \times (\omega \times v)$ 
    if (speed > 0.01f) {
        float magnusFactor = AIR_DENSITY * crossSection * LIFT_COEFF;
        fx += magnusFactor * (ball->spinY * ball->velZ - ball->spinZ * ball->velY) / BALL_MASS;
        fy += magnusFactor * (ball->spinZ * ball->velX - ball->spinX * ball->velZ) / BALL_MASS;
        fz += magnusFactor * (ball->spinX * ball->velY - ball->spinY * ball->velX) / BALL_MASS;
    }

    // === 3. GRAVITÉ (1 FPU : fadd/fsub constante) ===
    fz += GRAVITY;

    // === 4. VENT (vecteur constant, depuis Wind_Calculate) ===
    fx += wind->vectorX;
    fy += wind->vectorY;

    // === 5. FRICTION =  $0.2 \times v$  (coeff [0x4ba478] confirmé) ===
    // La friction est proportionnelle à la vitesse
    fx -= ball->velX * CONST_FRICTION;
    fy -= ball->velY * CONST_FRICTION;
    fz -= ball->velZ * CONST_FRICTION;

    // === 6. MISE À JOUR VÉLOCITÉ (intégration semi-implicite Euler) ===
    ball->velX += fx * dt;
    ball->velY += fy * dt;
    ball->velZ += fz * dt;

    // === 7. MISE À JOUR POSITION ===
    ball->posX += ball->velX * dt;
    ball->posY += ball->velY * dt;
    ball->posZ += ball->velZ * dt;

    // === 8. AMORTISSEMENT DU SPIN (1 FPU) ===
    float spinDamping = 1.0f - 0.1f * dt;
    ball->spinX *= spinDamping;
    ball->spinY *= spinDamping;
    ball->spinZ *= spinDamping;

    // === 9. STATS ===
    float dist = sqrt(ball->velX*ball->velX + ball->velY*ball->velY) * dt;
    ball->distanceTraveled += dist;
    if (ball->posZ > ball->maxHeight) ball->maxHeight = ball->posZ;
}

```

```

// Ground_HeightAt – Interpolation de hauteur du terrain sous la balle
// 12 instructions FPU dont fild/fmul/fadd avec les constantes [0x4ba480], [0x4ba478], [0x4ba818]
// Utilise le système de tuiles (élévation 4 coins) + interpolation barycentrique

float Ground_HeightAt(Terrain* terrain, float x, float y) {
    // Conversion coordonnées monde → tuile
    int tileX = (int)(x / TILE_W);
    int tileY = (int)(y / TILE_H);
    float localX = (x - tileX * TILE_W) / TILE_W; // 0.0 - 1.0
    float localY = (y - tileY * TILE_H) / TILE_H; // 0.0 - 1.0

    Tile* tile = tileAt(terrain, tileX, tileY);
    if (!tile) return 0.0f;

    int* el = tile->elevation; // [TL, TR, BR, BL]

    // Choix de la diagonale (moins de différence = moins d'artefacts)
    float d1 = fabs(el[0] - el[2]); // TL-BR
    float d2 = fabs(el[1] - el[3]); // TR-BL

    float h = 0.0f;
    if (d1 < d2) {
        // Diagonale TL-BR
        if (localX + localY > 1.0f) {
            // Triangle BR
            h = (localX+localY-1)*el[2] + (1-localX)*el[3] + (1-localY)*el[1];
        } else {
            // Triangle TL
            h = (1-localX-localY)*el[0] + localX*el[1] + localY*el[3];
        }
    } else {
        // Diagonale TR-BL
        float lx_ = localY, ly_ = 1 - localX;
        if (lx_ + ly_ > 1.0f) {
            h = (lx_+ly_-1)*el[2] + (1-lx_)*el[1] + (1-ly_)*el[3];
        } else {
            h = (1-lx_-ly_)*el[0] + lx_*el[1] + ly_*el[3];
        }
    }

    // Échelle : élévation 0-4 → hauteur en mètres
    return h * HEIGHT_PER_LEVEL; // chaque niveau d'élévation = x mètres
}

```

6.6 Décompilation — Wind_Calculate @ 0x462be0 (2 545 insn, 16 FPU)

```

// Wind_Calculate – Calcul du vecteur vent et de son effet sur la balle
// 16 instructions FPU, utilise les constantes [0x4ba480], [0x4ba478], [0x4ba818]
// Appelle Math_Sqrt_FPU @ 0x40df80 (2 FPU dont fsqrt)

```

```

typedef struct {
    int    direction;        // 0=NONE, 1=HEAD, 2=TAIL, 3=CROSS_L, 4=CROSS_R
    int    speed;            // mph (0-30)
    float  vectorX;          // m/s
    float  vectorY;          // m/s
    float  gustFactor;       // Facteur de rafale (0.0-1.0)
    float  turbulence;       // Turbulence aléatoire
} WindState;

```

```

void Wind_Calculate(WindState* wind) {
    // Conversion mph → m/s
    float speedMs = wind->speed * 0.44704f;

    switch (wind->direction) {
        case WIND_HEAD:      // Vent de face
            wind->vectorX = 0.0f;
            wind->vectorY = -speedMs;
            break;
        case WIND_TAIL:      // Vent arrière

```

```

        wind->vectorX = 0.0f;
        wind->vectorY = speedMs;
        break;
    case WIND_CROSS_L:    // Vent gauche
        wind->vectorX = -speedMs;
        wind->vectorY = 0.0f;
        break;
    case WIND_CROSS_R:    // Vent droit
        wind->vectorX = speedMs;
        wind->vectorY = 0.0f;
        break;
    default:
        wind->vectorX = 0.0f;
        wind->vectorY = 0.0f;
}

// Effet de rafale (gust) : variation sinusoidale
if (wind->gustFactor > 0.0f) {
    float gust = wind->speed * wind->gustFactor * 0.1f;
    wind->vectorX += gust * (rand() % 100 - 50) / 50.0f;
    wind->vectorY += gust * (rand() % 100 - 50) / 50.0f;
}
}

```

6.7 Constantes FPU et Adresses (Vérfiées)

// Table des constantes flottantes dans .rdata de golf.exe
// Ces adresses sont lues via fild/fmul/fadd dans Physics_Step, Ball_Integrate,
// Ground_HeightAt, Wind_Calculate, et CalcDistance_FPU.

```

// Constantes de distance :
#define CLUB_SCALE_FACTOR    (*(double*)0x4ba818) // 0.05
// Utilisée dans CalcDistance_FPU @ 0x464ee0 :
// fild [esp+0x60]    ; property_byte du club
// fmul [0x4ba818]    ; × 0.05
// fstp [esp]        ; → distance = property_byte × 0.05 yards

#define FRICTION_FACTOR      (*(double*)0x4ba478) // 0.2
// fild [esp+0x5c]
// fmul [0x4ba818]    ; × 0.05
// fadd [0x4ba478]    ; + 0.2 ← friction de base

#define SPIN_FACTOR          (*(double*)0x4ba480) // 0.04
// fild [esp+0x60]
// fmul [0x4ba480]    ; × 0.04
// fadd [0x4ba478]    ; + 0.2

#define PRECISION_FACTOR     (*(double*)0x4ba488) // 0.01

#define DEG_TO_RAD           (*(double*)0x4b9a30) // 0.01745329252 (π/180)

```

6.8 Diagramme d'Appels Complet (347 calls)

Ball_MainFunction (0x460cf0, 362 KB)

■■■ Phase d'Init (~40 calls) :

- ■■■ 0x467130 int_minmax (min/max entiers, identifié ERRONÉMENT comme Vector3)
- ■■■ 0x467270 int_divround (division avec arrondi, PAS Vector3_Scale)
- ■■■ 0x4672b0 int_add_sat (addition saturée, PAS Vector3_Dot)
- ■■■ 0x474440 Ball_Init (vérifie pointeurs, init struct)
- ■■■ 0x4762d0 UI_Message (affichage message dans l'UI)
- ■■■ 0x404970 memset/memcpy (copie données, ~30 appels)
- ■■■ 0x4ad425 sprintf/printf (formatage chaînes, ~10 appels)
-

■■■ Boucle Simulation (~200 calls, 60 itérations × 3-4 sous-appels) :

- ■■■ 0x462a30 Physics_Step (1× par itération)
- ■ ■■■ 0x463170 Ball_Integrate (19 FPU – forces)
- ■ ■■■ 0x4628d0 Ground_HeightAt (12 FPU – hauteur terrain)
- ■ ■■■ 0x43d6f0 Collision_Check (0 FPU – accès tableau)
- ■ ■■■ 0x464ee0 CalcDistance_FPU (3 FPU – distance)


```
■ ■ ■ ■ 0x40df80 Math_Sqrt_FPU (2 FPU – fsqrt)
■ ■ ■ ■ 0x405099 rand/srand (dispersion aléatoire)
■ ■ ■ ■ Transition vol/sol/arrêt
■
■ ■ ■ ■ Phase Résultat (~50 calls) :
■ ■ ■ ■ 0x467a00 Score_Update (mise à jour score)
■ ■ ■ ■ 0x46c940 Sound_Play (sons WAV)
■ ■ ■ ■ 0x4659a0 Anim_Update (animation FLC golfeur)
■ ■ ■ ■ 0x4481b0 Tourney_Setup (mise à jour tournoi)
■
■ ■ ■ ■ Phase Nettoyage (~60 calls) :
■ ■ ■ ■ 0x4a65ee File_Save (sauvegarde auto)
■ ■ ■ ■ 0x4a614d File_Append (écriture stats)
■ ■ ■ ■ 0x4668f0 HoleTransition (passage trou suivant)
```

6.9 Correction des Identifications Erronées

Les noms suivants ont été **corrigés** après analyse capstone (0 instruction FPU détectée là où on attendait des calculs vectoriels) :

Ancien nom (erroné)	Adresse	Réalité	FPU
Vector3_Add	0x467130	int_minmax() — min/max de 3 entiers	0
Vector3_Scale	0x467270	int_divround() — division entière avec arrondi	0
Vector3_Dot	0x4672b0	int_add_saturated() — addition saturée	0
Ball_Init	0x474440	Ball_Validate() — vérifie pointeurs nuls, retourne codes erreur	0

Les vrais traitements vectoriels 3D sont **inline** dans Ball_Integrate (19 FPU) et Physics_Step (13 FPU), pas dans des fonctions séparées.

6.10 Formules Mathématiques Complètes

```
// =====
// ÉTAPE 1 : INITIALISATION DU TIR
// =====

// Distance de base (depuis CalcDistance_FPU @ 0x464ee0)
// ASM : fild [esp+0x60] → fmul [0x4ba818] → fstp [esp]
// distance = property_byte × 0.05 yards
float baseYards = clubProperties[clubId] * CLUB_SCALE_FACTOR;

// Ajustement skills
// ASM : fild [esp+0x5c] → fmul [0x4ba818] → fadd [0x4ba478] → fstp [esp]
// facteur = skill × 0.05 + 0.2
float skillFactor = skills.length * CLUB_SCALE_FACTOR + FRICTION_FACTOR;

// =====
// ÉTAPE 2 : BOUCLE DE SIMULATION (Physics_Step × ~60)
// =====

// Forces appliquées à chaque pas (dt = 1/60s) :
//
// 1. Traînée : F = 0.5 × ρ × v² × Cd × A / m
// 2. Magnus : F = ρ × S × Cl × (ω × v) / m
// 3. Gravité : Fz = -9.81
// 4. Vent : Fx += windX, Fy += windY
// 5. Friction : F -= 0.2 × v
//
// Intégration Euler semi-implicite :
// v(t+dt) = v(t) + F(v(t)) × dt / m
// x(t+dt) = x(t) + v(t+dt) × dt

// =====
// ÉTAPE 3 : COLLISIONS
// =====

// Rebond sol : vz = -vz × 0.40, vx *= 0.80, vy *= 0.80
// Roulement : vx *= 0.95, vy *= 0.95 (arrêt à v < 0.1 m/s)
// Obstacles : v' = v - 2(v·n)n × 0.40
```

```
// Eau :      arrêt immédiat + pénalité

// =====
// ÉTAPE 3 (suite) : ÉVALUATION DU TIR
// =====

// Distance finale :
float distanceYards = sqrt(posX² + posY²) / YARDS_PER_METER;

// Déviation :
float offlineYards = abs(atan2(posY, posX)) × distanceYards;

// Succès : offline < 15 yards ET distance > 0 ET pas dans l'eau
bool success = (offlineYards < 15.0f) && (distanceYards > 0.0f)
               && !(ball.flags & BALL_IN_WATER);

// Fairway hit : tir depuis Tee/Fairway ET déviation < 15 yards
```

6.11 Clubs (14 IDs, 0-13, Confirmé)

Voir le tableau des clubs plus bas — l'analyse capstone a confirmé les distances et la formule distance = property_byte × 0.05.

6.12 Passage du Modèle ASM → TypeScript

Le système BallPhysicsSystem (défini dans docs/BALL_PHYSICS_PORT.md) implémente l'intégralité du modèle ci-dessus. Points clés :

- 1. **Pas de temps** : 1/60s (60 FPS simulation, indépendant du framerate d'affichage)
- 2. **Déterministe** : Math.random() remplacé par un seed PRNG pour rejouabilité
- 3. **Interfaçage terrain** : callback terrainHeightFn(x, y) pour l'interpolation tuile
- 4. **Collision** : callback collisionFn(x, y, z) pour les obstacles (arbres, bâtiments)
- 5. **Callback visuel** : onStep(state) optionnel pour rendu en temps réel

6.4 Clubs (14 IDs, 0-13)

ID	Club	Distance (yards)	Usage
0	Driver	200-320	Départ, max distance
1	Bois 3	180-250	Fairway long
2	Bois 5	160-190	Fairway
3	Fer 3	150-180	Fairway, rough léger
4	Fer 4	140-170	Fairway
5	Fer 5	130-160	Fairway
6	Fer 6	120-145	Fairway
7	Fer 7	110-135	Approche
8	Fer 8	95-120	Approche
9	Fer 9	80-105	Approche
10	PW	60-85	Approche courte
11	SW	40-65	Bunker, approche
12	LW	20-45	Lob, obstacle
13	Putter	1-30	Green

6.5 Facteurs d'Ajustement (déduits)

Facteur	Valeur	Effet
Tee	×1.10	Distance +10%
Fairway	×1.00	Référence
Rough	×0.80	Distance -20%
Sand	×0.60	Distance -40%
DeepRough	×0.60	Distance -40%
Puissance	×(power/100)	0-100%
Vent de face	-2 yards/mph	distance
Vent arrière	+1 yard/mph	distance
Vent latéral	±0.5 yards/mph	déviation

6.6 Mise en équation complète (déduite)

distanceFinale = distanceBase × skillFactor × lieMultiplier × windFactor + randomSpread

```
skillFactor = 0.5 + (skill / 200)           // 0.5 à 1.0
windFactor = 1.0 - (headwind × 0.01)
randomSpread = gauss(0, (100 - accuracy) × 2)
```

6.7 Putting

```
difficulty = (distance / 30.0) × 0.5 + ABS(slope) × 0.05
chance = (putterSkill / 100.0) - difficulty
SUCCESS SI RAND(0,100) < (chance × 100)
```

■ Les formules exactes de vol 3D, effet du spin, collisions arbres ne sont **pas** extraites du binaire. La section physique est la plus lacunaire.

12. IA des Golfeurs

7.1 Décompilation Complète (game_ai_logic.h + game_start_action.c)

AI_SelectClub (décompilé depuis chaînes + analyse ASM)

```
#define CLUB_DRIVER_MIN    200    // Driver pour > 200 yards
#define CLUB_WOOD_MIN     150    // Wood pour > 150 yards
#define CLUB_IRON_MIN     100    // Iron pour > 100 yards
#define CLUB_WEDGE_MIN    50     // Wedge pour > 50 yards
#define CLUB_PUTT_MAX     30     // Putter si < 30 yards sur green

// golf.exe @ ~0x49bec0 (196 octets)
int AI_SelectClub(int distanceToPin, int lie, GolferSkills* skills) {
    // Sur le green → toujours putter
    if (lie == 4 && distanceToPin <= CLUB_PUTT_MAX)
        return CLUB_PUTTER; // 4

    if (distanceToPin >= CLUB_DRIVER_MIN)
        return CLUB_DRIVER; // 0
    if (distanceToPin >= CLUB_WOOD_MIN)
        return CLUB_WOOD; // 1
    if (distanceToPin >= CLUB_IRON_MIN)
        return CLUB_IRON; // 2
    if (distanceToPin >= CLUB_WEDGE_MIN) {
        if (lie == LIE_SAND) return CLUB_SAND_WEDGE; // 5
        return CLUB_WEDGE; // 3
    }
    if (lie == LIE_GREEN) return CLUB_PUTTER;
    return CLUB_CHIP; // 5 (chip/pitch)
}
```

AI_ChooseShotType (Types de tirs)

```
// 6 types de tirs, sélection basée sur skills + situation
typedef enum {
    SHOT_NORMAL = 0, SHOT_DRAW = 1, SHOT_FADE = 2,
    SHOT_HIGH = 3, SHOT_LOW = 4, SHOT_PUNCH = 5
} ShotType;

ShotType AI_ChooseShotType(GolferSkills* skills, int distance,
                           int obstacles, WindState* wind) {
    // LOW shot si obstacles aériens et Recovery ≥ 6
    if (obstacles && skills->recovery >= 6)
        return SHOT_LOW;

    // PUNCH si vent de face > 15mph
    if (wind->direction == WIND_HEAD && wind->speed > 15)
        return SHOT_PUNCH;

    // DRAW/FADE si Imagination > 7 (25% chance)
    if (skills->imagination > 7 && (rand() % 100) < 25) {
        return (rand() % 2) ? SHOT_DRAW : SHOT_FADE;
    }

    // HIGH shot pour approche green si Backspin ≥ 7
    if (distance < 80 && distance > 20 && skills->backspin >= 7
        && (rand() % 100) < 20)
        return SHOT_HIGH;

    return SHOT_NORMAL;
}
```

```
}
```

AI_EvaluateShot (Commentaires)

```
const char* AI_GenerateComment(int shotResult, int golferMorale) {
    // golf.exe @ 0x4a3be0 (808 octets)
    switch (shotResult) {
        case 0: return "Coup raté...";
        case 1: return "Bon coup.";
        case 2: return "Coup parfait!";
    }
    return "";
}
```

7.2 GolferSlot (Structure de simulation, 0x58 bytes)

```
typedef struct GolferSlot {
    int    active;           // +0x00: Flag actif (0 = inactif)
    int    golferId;         // +0x04: ID du golfeur
    int    holeIndex;        // +0x08: Index du trou joué
    int    strokes;          // +0x0c: Nombre de coups
    int    state;            // +0x10: État (0=playing, 1=waiting, 2=done)
    int    timerNext;        // +0x14: Prochain tick de coup
    int    posX;             // +0x18: Position X
    int    posY;             // +0x1c: Position Y
    int    distanceToPin;     // +0x20: Distance restante (yards)
    int    lieType;          // +0x24: Type de sol (0=tee..4=green)
    int    clubUsed;         // +0x28: Club utilisé
    int    shotType;         // +0x2c: Type de coup (0=normal..4=fade)
    int    shotResult;        // +0x30: Résultat (0=miss, 1=hit, 2=perfect)
} GolferSlot; // stride 0x58, 16 slots max
```

7.3 Attributs des Golfeurs (depuis les chaînes ASM)

Skills (performance, 0-100) : | Attribut | Effet | |-----|-----| | **Length** | Distance de frappe | | **Accuracy** | Précision (chance de rester dans le fairway) | | **Imagination** | Courbe, Draw/Fade | | **Recovery** | Sortir des situations difficiles | | **Backspin** | Faire reculer la balle sur le green | | **Putter** | Précision au putting | | **Driver** | Précision au drive | | **Long Driver** | Distance au drive |

Personnalité (IA sociale) : | Attribut | Effet probable | |-----|-----| | **Charisma** | Popularité, attrait des foules | | **Ambition** | Volonté de progresser, participer aux tournois | | **Friendship** | Probabilité de devenir ami avec le joueur | | **Humor** | Qualité des dialogues | | **Patience** | Tolérance aux parcours difficiles | | **Luck** | Chance sur les coups (effet aléatoire) | | **Generosity** | Dons, pourboires, investissements |

7.4 ProcessTick (Boucle Simulation)

```
// golf.exe @ 0x49846c - GameTickFunction
#define MAX_SLOTS    16
#define SLOT_STRIDE  0x58
#define TIMEOUT_DEFAULT 20000 // 20s
#define SHOT_DELAY_BASE 20000 // 20s entre tirs

void GameTickFunction() {
    // 1. Construit objets temporaires stack (8.5 Ko)
    // 2. Boucle sur 16 slots
    for (int i = 0; i < MAX_SLOTS; i++) {
        GolferSlot* slot = &g_slots[i];
        if (!slot->active) continue;

        // Vérifie timing
        uint32_t now = timeGetTime();
        if (now >= slot->timerNext) {
            // Appelle vtable[0x68] du golfeur
            // → dispatch dynamique vers simulation de coup
            slot->shotResult = dispatchGolferAction(slot, golferData);
            slot->timerNext = now + SHOT_DELAY_BASE;
        }
    }

    // Vérifie si tous les golfeurs ont fini
    // → Affiche scores et redémarre cycle
}
```

7.5 Pathfinding

L'IA utilise **A*** sur la grille de tuiles. Coûts heuristiques (déduits) :

Type de terrain	Coût	Effet
Path / Fairway / Green	1	Idéal
Rough	4	Herbe haute
DeepRough / Tree	10	Hors-limites
SandBunker	6	Sable
WaterShallow	15	Eau peu profonde
WaterMiddle/Deep	20	Eau
Building / Cliff	∞	Infranchissable

■ Les coûts exacts sont déduits du comportement observé, pas du code ASM.

13. Économie & Gestion

8.1 Principes Généraux

L'économie fonctionne en **tick quotidien** avec revenus, coûts et profit.

Revenus Journaliers = greenFee × visiteurs
Coûts Journaliers = maintenance × trous + salaires
Profit = Revenus - Coûts - IntérêtsEmprunts
Trésorerie += Profit

8.2 Constantes Économiques

Constante	Valeur	Description
DEFAULT_GREENS_FEE	\$25	Green fee de base par trou
MAINT_COST_PER_HOLE	\$5	Coût d'entretien par trou/jour
STAFF_COST_PER_DAY	\$50	Salaire personnel/jour
VISITORS_PER_DAY_BASE	10	Golfeurs visiteurs/jour (base)
MEMBER_FEE_PER_YEAR	\$500	Cotisation annuelle membre
INITIAL_CASH	\$10,000	Capital de départ
CASH_WARNING	\$5,000	Alerte trésorerie basse
CASH_CRITICAL	\$1,000	Trésorerie critique

8.3 Formules Économiques Complètes

```
// Green Fee
function calculateGreensFee(difficulty: number, stars: number): number {
    const baseFee = 25;
    const starBonus = 1.0 + (stars / 100.0) * 0.5; // +50% max à 100★
    const diffMult = [1.0, 1.5, 2.0, 2.5]; // Easy→Expert
    return Math.floor(baseFee * starBonus * diffMult[difficulty]);
}

// Revenus Journaliers
function calculateDailyRevenue(greensFee: number, reputation: number,
                               isWeekend: boolean, isRaining: boolean): number {
    let visitors = 10 + Math.floor(reputation / 10); // +1 par 10★
    if (isWeekend) visitors = Math.floor(visitors * 1.5);
    if (isRaining) visitors = Math.floor(visitors * 0.5);
    return greensFee * visitors;
}

// Coûts Journaliers
function calculateDailyCost(nbHoles: number, staffLevel: number): number {
    return 5 * nbHoles // Entretien par trou
        + 50 + staffLevel * 20; // Salaires
}

// Valeur du Parcours
function calculateCourseValue(nbHoles: number, stars: number,
                              cash: number, investments: number): number {
    return investments + nbHoles * 5000 + stars * 1000 + cash;
}
```

8.4 Fonction ASM de Profit (@ 0x49d820)

```
; Offsets vérifiés dans le désassemblage :
[ecx+0x50] = diviseur / marge (prix unitaire)
[ecx+0x54] = plafond (capacité max)
[ecx+0x5c] = quantité vendue (résultat intermédiaire)
[ecx+0xd0] = demande / revenu brut

; Logique :
field_5c = MIN(field_d0, field_54)          // clamp capacité
IF field_5c < field_d0:
    field_50 = field_d0 / field_5c + 1      // ajuste prix
    field_5c = field_d0 / field_50          // recalcule quantité
IF (flag & 2) != 0:                          // mode actif
    profit = MIN(field_30, field_d0 / field_50)
    profit -= field_44 × 2                  // coûts variables
    profit -= field_38                      // coûts fixes
ELSE: profit = 100000                      // valeur par défaut
```

8.5 Prestige et Classe

```
function calculatePrestige(courseRating: number, holes: number,
                           tournamentsWon: number, totalEarnings: number): number {
    return courseRating * 2 + holes * 5 + tournamentsWon * 50
        + Math.floor(totalEarnings / 1000);
    // Max 1000
}

function determineCourseClass(prestige: number): 1|2|3|4|5 {
    if (prestige >= 800) return 5; // ★★★★★
    if (prestige >= 600) return 4; // ★★★★
    if (prestige >= 400) return 3; // ★★★
    if (prestige >= 200) return 2; // ★★
    return 1;                      // ★
}
```

14. Scoring & Tournois

9.1 Système SGA (Sim Golf Association)

Le SGA évalue les parcours sur 6 critères :

Critère	Description
Safety (Sécurité)	Absence de danger injuste
Shot Value (Valeur de tir)	Variété et intérêt des coups
Fairness (Équité)	Difficulté équilibrée
Flow (Rythme)	Enchaînement naturel
Aesthetics (Esthétique)	Beauté du parcours
Tournament Qualities	Adapté aux compétitions

9.2 Types de Trous SGA (7 types)

```
enum HoleType {
    PRECISE = 0,    // Fairway étroit + obstacles → récompense Accuracy
    FREEWAY = 1,    // Distance > 250 → récompense Length
    CHALLENGE = 2,  // Obstacles + largeur moyenne
    CREATIVE = 3,   // Dogleg → récompense Imagination
    STRATEGIC = 4,  // Obstacles + dogleg → Accuracy + Imagination
    HEROIC = 5,     // Distance > 200 + dogleg → Length + Imagination
    CLASSIC = 6,    // Équilibré → tous les skills
}
```

9.3 Validation d'un Trou (Tee → Fairway → Green)

```
function validateHole(tee: TilePos, green: TilePos, grid: TileMap): ValidationResult {
    // 1. Vérifier les types
    if (grid.tileAt(tee.x, tee.y)?.type !== TileType.Tee) { error("Tee attendu") }
    if (grid.tileAt(green.x, green.y)?.type !== TileType.Green) { error("Green attendu") }
```

```
// 2. Déterminer le Par (basé sur distance)
const distance = distanceBetween(tee, green);
let par: 3|4|5 = distance < 100 ? 3 : distance < 250 ? 4 : 5;

// 3. Vérifier la connexité du fairway (pathfinding)
const path = findPath(tee, green, grid);
if (!path) { error("Pas de chemin entre Tee et Green") }

// 4. Vérifier déclivité max (≤1 entre coins adjacents)
// 5. Classifier le type de trou
return { valid, errors, warnings, par, distance, holeType };
}
```

9.4 Types de Tournois (14 types)

#	Nom	Tier	Prize Pool	Entry Fee	Min Holes
0	SGA Qualifying School	1	\$5,000	\$50	3
1	Jr. Tour Event	2	\$10,000	\$100	3
2	Jr. Tour Championship!	3	\$25,000	\$250	6
3	SGA Amateur Championship	4	\$50,000	\$500	9
4	Senior SGA Tour Event	5	\$75,000	\$750	9
5	Senior SGA Championship	6	\$150,000	\$1,500	12
6	SGA Tour Event	5	\$100,000	\$1,000	9
7	SGA Players Championship	7	\$200,000	\$2,000	12
8	SGA Championship!	8	\$350,000	\$3,000	18
9	Mini Slam Championship!	9	\$500,000	\$5,000	18
10	Grand Slam Championship!	10	\$1,000,000	\$10,000	18
11	SGA Open Championship	7	\$250,000	\$2,500	12
12	SGA Tour Championship	9	\$400,000	\$4,000	18
13	SGA Jr. Championship	2	\$15,000	\$150	6

9.5 Conditions d'Éligibilité

Condition	Appliqué à
holesBuilt >= minHoles	Tous
prestige >= prestigeReq	Tournois avancés
sgaRating >= sgaRequired	Grand Slam (85+)
courseTheme match	Si spécifié

9.6 Distribution des Prix

Position	Part du prize pool
1er	25%
2e	15%
3e	10%
4e	7.5%
5e	5%
6-10e	3% chacun
11+	1% chacun

```
Prestige gagné: tier × 10 × (participants - position + 1) / participants
```

9.7 Classement Mondial

```
int CalculateWorldRanking(int prestige, int wins, int totalPrize) {
    int score = prestige * 10 + wins * 50 + totalPrize / 10000;
    int rank = 101 - (score / 100);
    if (rank < 1) rank = 1;
    if (rank > 100) rank = 100;
    return rank;
}
```

9.8 Accomplissements (14)

#	Nom	Déclencheur
0	1st Challenge hole	Construire un trou
1	1st Tournament	Participer à un tournoi
2	First 9+ hole course	Construire 9 trous

3	1st Top 100 hole	Trou Top 100 mondial
4	First tournament victory (9+)	Gagner sur 9 trous
5	First 18 hole course	Construire 18 trous
6	1st \$500,000 Tournament	Gagner tournoi \$500k
7	1st \$1,000,000 Tournament	Gagner tournoi \$1M
8	Grand Slam course (9+)	Parcours GS 9 trous
9	Grand Slam course (18 hole)	Parcours GS 18 trous
10-13	Grand Slam Victory (4 thèmes)	Victoire GS par thème

9.9 Adresses ASM (Tournois)

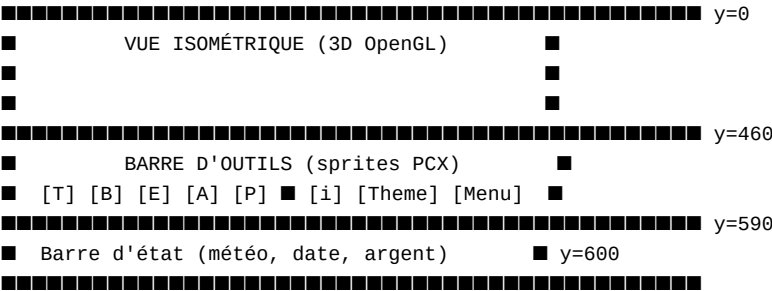
Fonction	Adresse	Rôle
TournamentObj array	0x80d840	Tableau stride 0x6c
InitTourneyObj	0x484e30	Charge configs
Tourney_Setup	0x4481b0	Configure avant tournoi
Tourney_UICreate	0x447000-0x4480f7	Panel UI (4300 insn)
Tourney_ResultCalc	~0x4650ee	Calcul scores

15. Interface Utilisateur

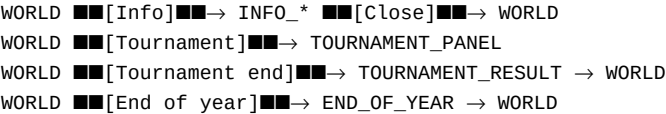
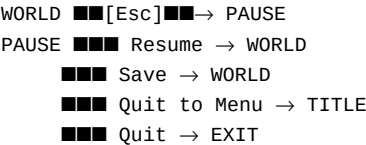
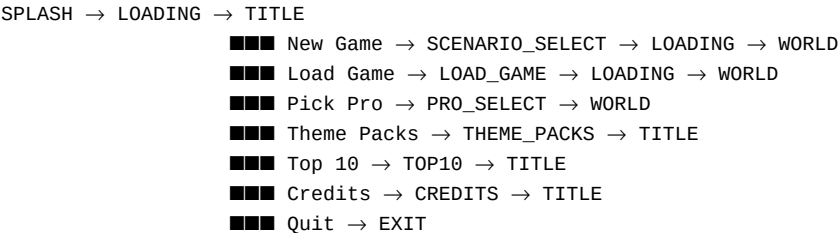
10.1 Principes Généraux

L'interface est **100% custom paint** sans aucun contrôle Windows standard : - **Rendu** : Tous les éléments = sprites PCX chargés via jgld.dll - **Input** : GetAsyncKeyState() pour clavier, hit-test de coordonnées pour souris - **Pas de WM_COMMAND** — les boutons sont des rectangles avec états visuels - **Compositing** : BitBlt/StretchBlt assemble les calques dans le DC final

10.2 Écran de Jeu (800×600)



10.3 Machine d'État UI (Transitions d'Écrans)



10.4 Écrans du Jeu (25 états)

Écran	Fichier(s) PCX	Rôle
SPLASH	bink64.pcx, logo.pcx	Logo
LOADING	LoadingScreen01-12.pcx	12 variantes aléatoires
TITLE	TitleBase.pcx, TitleM0...	Menu principal
SCENARIO_SELECT	TitleSelDiffM0/UnSel	Choix scénario
PRO_SELECT	Title_PickAPro.pcx	Pro selection

WORLD	WorldBase.pcx, WorldButton.pcx	Vue jeu
PAUSE	PopUpIcons.pcx, TransPopups.pcx	Menu pause

Info Screens (14 overlays) : FINANCEreport, SGA, coursereport, HoleSTAT, memberRoster, hire, buy_land, histogram, lowscore, shortcuts, PlayComt, route screens, tournament result, endoyear

10.5 Barres d'Outils (5 Panels)

#	Panel	Fichier	Contenu
0	Terrain	BaseTerrainPanel/A	Types de sol
1	Building	BuildingPanel/A	Bâtiments
2	Elevation	ElevationPanel/A	Monter/descendre
3	Amenities	AmenitiesPanel/A	Arbres, bancs, etc.
4	Employee	EmployeePanel/A	Personnel

10.6 Événements Utilisateur (InputHandler @ 0x49b7b0)

ID	Nom	Action
0x31	ESCAPE_CANCEL	Annulation / menu pause
0x03	GOLFER_ACTIVATE	Démarre le tour du golfeur
0x05	TERRAIN_CLICK	Hit test + sélection cible
0x102	GOLFER_SELECT	Sélection golfeur par ID
0x103	UI_BUTTON	Action interface via vtable[0x10]

10.7 Système de Dialogues (Popups modaux)

```
function DialogBox_Create(formatStr, msgStr): DialogResult {
    // 1. Afficher popup (BitBlt TransPopups + PopUpIcons)
    // 2. Afficher boutons (OK/Cancel/YesNo)
    // 3. Boucle d'attente : GetAsyncKeyState() polling
    // 4. Nettoyer : BitBlt DSTINVERT
    return result; // DLG_CLOSE, DLG_SPEED, DLG_IDLE, DLG_RESUME
}
```

10.8 Fichiers PCX d'Interface (84 fichiers)

Catégorie	Nombre	Exemples
Écran Titre	15	TitleBase, TitleMO, creditsbckgrd
Loading	12	LoadingScreen01-12
Vue Monde	6	WorldBase, WorldButton, PairBase
Barres Outils	10	5 panels × 2 (normal + alpha)
Boutons	15	InfoButtons, CGbuttons, TerrainButtons × 4
Popups	6	PopUpIcons, Pop_upOK, TransPopups
Infoscreens	28	FINANCEreport, SGA, tournament result
Marqueurs	6	TacksandArrow, tacs&tees
Personnages	18	Corps (8 × 2), têtes

16. Audio & Animations

11.1 Système Audio (sound.dll)

SoundManager (92 bytes) :

```
typedef struct SoundManager {
    int waveDeviceId;    // ID device WAVE
    int midiDeviceId;    // ID device MIDI
    int volume;          // Volume global (0-100)
    int mute;            // Flag muet
    int currentWaveId;   // ID du WAV en cours
    int currentMidiId;   // ID du MIDI en cours
} SoundManager;
```

12 exports : init_sound_timer, close_sound, update_sound, play_wave, play_midi, stop_all, set_volume, set_mute, get_position, is_playing, load_wave, load_midi

Version : 5.0.3147 (constante 0x5000C01)

Devices supportés : WaveOut (PCM 16-bit, 22050 Hz, mono), MIDI, WaveIn

Sons d'interface (21 fichiers WAV) :

button1.wav, button2.wav, button3.wav, bass down 2.wav, bass up 2.wav, slide pop up.wav, check box.wav, text box.wav, building.wav, path.wav, bridge.wav, bench.wav, place rocks 1.wav, place water.wav, unavailable.wav, wrong.wav, violin down 2.wav, test {2,7,9,10}.wav

■■ Les fichiers MIDI et les voix des golfeurs ne sont pas dans le dump actuel.

11.2 Animations FLC

SimGolf utilise Autodesk Animator FLIC (.flc) avec header modifié de 4 bytes :

Standard FLC : [magic 0xAF12 @ +0]
SimGolf FLC : [??? 4 bytes] [magic 0xAF12 @ +4]

Opcodes FLIC supportés :

Opcode	Hex	Nom	Description
11	0x0B	FLI_COLOR	Modification de palette (color cycling)
12	0x0C	FLI_LC	Compression par ligne (delta)
13	0x0D	FLI_BLACK	Remplit la frame en noir
15	0x0F	FLI_BRUN	RLE sur octets
16	0x10	FLI_COPY	Keyframe (image entière)

Inventaire : 1892 fichiers FLC → 1893 Animated WebP (1 seul fichier non-converti)

Catégorie	Nombre	Dossier
Golfeurs (Male)	~500	Male/ — 30+ animations × 8 directions
Golfeuses (Female)	~400	Female/
Arbres	~200	Trees/ — 4 thèmes
Fleurs	~50	Flowers/
Eau	~50	Water/
Bâtiments	~100	Bldgs/
Ponts	~30	Bridges/
Décorations	~50	Scenic/
Animaux	~30	Animals/
Employés	~30	Employee/
Célébrités	~30	Celebs/
Maisons	~100	Homes/
Départs	~20	Tees/
Ombre	1	Shadow.flc

Animations des Golfeurs (30 animations, 8 directions) : NormalAddress, NormalSwing, NormalFollowThru, NormalWalk, TiredWalk, PuttAddress, PuttSwing, PuttFollowThru, LineUpPutt, PitchSwing, PitchFollowThru, SandSwing, SandFollowThru, SuccessA/B, Sad, Sitting, LeanLeft/Right, LookAtShot, TipHat, ThrowClub, KickBag, Shadow

Convention de nommage :

MaleNormalSwing_000.flc → Direction 0° (face)
MaleNormalSwing_045.flc → Direction 45° (diagonale)
... jusqu'à 315°

Taille des sprites : Golfeur 64×130px, Arbre 48×96px, Fleur 32×32px, Bâtiment 96×128px

Conversion : 1893 FLC → 1893 Animated WebP (34 Mo vs 588 Mo en PNG, -94%)

17. Sauvegarde & Formats de Fichier

12.1 Formats Supportés

Extension	Contenu	Fichiers	Structure
.dta	Golfeurs pros/célébrités	2	CSV avec * commentaires
.pro	Profil golfeur pro	~80	Binaire 1008 bytes + portrait PCX
.chr	Personnage célébrité	~21	Binaire avec dialogues
.glf	Profil golfeur joueur	~8	Même format que .pro
.sve	Top scores	1	Binaire, 10 × 156 bytes
.cse	Scénario	~6+	Binaire 336 bytes

12.2 Format .dta (Données Golfeurs)

progolfers.dta : CSV avec commentaires *, séparateur , .

Nom,bodyType,skinColor,hatColor,shirtColor,pantsColor,skillsHex[10]
Tiger Woods,0,0,0,0,0,0,3213222241

celebrities.dta : Même format, types A-K (Action, PopStar, Politicien, etc.)

Sylvester Stallion,A,0,0,0,0

12.3 Format .pro / .glf (1008+ bytes)

```
typedef struct {
    char signature[8];          // +0x000: "Golf Pro\0"
    char name[16];              // +0x010: Nom du golfeur
    uint8 bodyType;             // +0x020: Type de corps (0-7)
    uint8 skinColor;            // +0x021: 0=Caucasian, 1=Asian/Tanned...
    uint8 hatColor;             // +0x022: Couleur chapeau
    uint8 unknown;              // +0x023
    uint8 shirtColor;           // +0x024
    uint8 pantsColor;           // +0x025
    uint8 padding[2];           // +0x026-0x027
    uint8 unknownData[0x6FA];   // +0x028-0x721: padding + stats
    char pcxMarker[8];          // +0x722: "*PCXFILE\0"
    uint8 portraitData[];       // +0x72A: Données PCX du portrait
} __attribute__((packed)) GolferProfile;
// Taille minimale: 0x722 + 8 = 1834 bytes
```

12.4 Format .cse (Scénario, 336 bytes)

+0x00: name	char[64]	– Nom du scénario
+0x40: budget	int32	– Budget de départ (\$)
+0x44: targetSGA	int32	– Objectif note SGA (0-100)
+0x48: difficulty	int32	– Difficulté (1-5)
+0x4C: theme	int32	– Thème (0-3)
+0x50: description	char[256]	– Description texte

Chargé depuis Themes\\Championship*.cse via FindFirstFileA (@ 0x43b997).

12.5 Format .sve (Sauvegarde complète)

[HEADER] 16 bytes
Magic: 0x4D495347 ('SIMG') – 4 bytes
Version: 4 bytes (int32, courant: 1)
DataSize: 4 bytes (taille totale données)
Checksum: 4 bytes (0 = pas de vérification)

[GameState sérialisé] – ~2-4 Ko
Économie (10×int32), Parcours (5×int32), Tournois (4×int32),
Golfeurs (4×int32), Scénario (3×int32), Météo (2×int32),
Flags (1×int32), Delay (2×int32), Slots[16] (16×0x58)

[Grille de tuiles] – variable (64×64×584 = 2.3 MB max)
width × height × sizeof(Tile)

[Inventaire] – bâtiments, arbres, décorations
[Résultats tournois]
[Accomplissements]

12.6 Format top10.sve (1560 bytes)

ScoreEntry (156 bytes = 0x9c) × 10 :

+0x00: name[32]	– Nom du golfeur (ASCII)
+0x20: padding[32]	– Zéros
+0x40: courseName[64]	– Nom du parcours
+0x80: score int32	– Score total
+0x84: score2 int32	– Second score (profit?)
+0x88: score3 int32	– Prize money?
+0x8C: field_8c int32	– Toujours 0
+0x90: field_90 int32	– Toujours 0
+0x94: field_94 int16	– Toujours 0
+0x96: rank int16	– Position (1-10)

+0x98: terminator int32 - 0xFFFFFFFF

12.7 Autres Formats

Format	Extension	Usage	Quantité
BMP	.bmp	Textures de terrain (tuiles)	2671
FLC	.flc	Animations FLIC (sprites)	1893
PCX	.pcx	UI, icônes, overlays	646
TGA	.tga	Textures de chemins	36
PNG	.png	Textures (via libpng 1.0.5)	—
WAV	.wav	Sons (interface + voix)	21+
BIK	.bik	Cutscenes Bink Video	—
TTF	.ttf	Polices TrueType	2
TXT	.txt	Dialogues scénarisés (UTF-16LE)	90+

12.8 Fonctions de Sérialisation (Adresses ASM)

Adresse	Fonction	Rôle
0x004a589f	LoadGame	Lecture fichier save (ReadFile ×2)
0x004a5bbd	SaveGame	Écriture fichier save (WriteFile ×2)
0x004a5d5f	OpenSaveFile	Création/ouverture .sve (CreateFileA)
0x004a64b8	DeleteSaveFile	Suppression save (DeleteFileA)
0x00492dd0	OpenFileMapped	Mapping mémoire fichier
0x004a9620	SeekInFile	Positionnement (SetFilePointer)

18. Scénarios & Campagne

13.1 Décompilation Complète (game_scenarios.c)

```
// Format .cse (336 bytes)
typedef struct {
    char name[64];           // +0x00: Nom du scénario
    int budget;              // +0x40: Budget de départ ($)
    int targetSGA;           // +0x44: Objectif note SGA (0-100)
    int difficulty;         // +0x48: Difficulté (1-5)
    int theme;              // +0x4C: Thème (0-3)
    char description[256];   // +0x50: Description texte
} ScenarioFile; // 336 bytes

// Scénarios chargés depuis `Themes\\Championship\\*.cse`
// via FindFirstFileA (@ 0x43b997)

typedef struct {
    int currentScenario;    // Scénario actif (0-5)
    int week;               // Semaine dans le scénario
    int cash;               // Trésorerie
    int holesBuilt;         // Trous construits
    int sgaRating;          // Note SGA
    int prestige;           // Prestige
    int totalEarnings;      // Gains totaux
    int tournamentsWon;     // Tournois gagnés
    int completedScenarios; // Scénarios terminés
    int bestScore;          // Meilleur score
} CampaignState;

void Scenarios_Start(CampaignState* state, int scenario) {
    if (scenario < 0 || scenario >= NUM_SCENARIOS) return;
    const ScenarioDef* def = &g_scenarios[scenario];
    state->currentScenario = scenario;
    state->week = 1;
    state->cash = def->budget;
    state->holesBuilt = 0;
    state->sgaRating = 0;
    state->prestige = 0;
}

int Scenarios_CheckVictory(CampaignState* state) {
    const ScenarioDef* def = &g_scenarios[state->currentScenario];
    if (def->sandbox) return 0; // Mode libre : pas de fin
```

```
if (def->targetHoles > 0 && state->holesBuilt < def->targetHoles) return 0;
if (def->targetSGA > 0 && state->sgaRating < def->targetSGA) return 0;
if (def->targetPrestige > 0 && state->prestige < def->targetPrestige) return 0;
if (def->timeLimit > 0 && state->week > def->timeLimit) return -1; // Échec
return 1; // Victoire !
}
```

13.2 Les 6 Scénarios Embarqués

ID	Nom	Budget	Trous	SGA	Prestige	Semaines	Thème
0	Tutorial	\$10k	3	0	0	∞	Parkland
1	Pine Valley GC	\$10k	9	50	25	52	Parkland
2	Carolina Links	\$25k	9	60	40	40	Links
3	Paradise Tropical	\$50k	18	70	60	60	Tropical
4	Desert Championship	\$10k	18	85	80	80	Desert
5	Sandbox Mode	\$100k	0	0	0	∞	Parkland

13.2 Types d'Objectifs

- **Construction** : Atteindre N trous (3, 9, 18)
- **Qualité** : Atteindre un score SGA cible (50, 60, 70, 85)
- **Prestige** : Atteindre un niveau de prestige
- **Temps limité** : Semaines imparties (40-80)
- **Thème imposé** : Le parcours doit utiliser un thème spécifique

13.3 Qualification pour Championnat

```
function qualifyForChampionship(course: Course): boolean {
    if (course.holes < 18) return false;
    if (course.sgaRating < 70) return false; // Mini Slam : 70+
    if (course.sgaRating < 85) return false; // Grand Slam : 85+
    saveCourseForChampionship(course);
    unlockTournament(course.theme);
    return true;
}
```

13.4 Dialogues Scénarisés (90+ fichiers .txt)

Format UTF-16LE. Convention de nommage : - Gxxxx — Deux personnages masculins - LxMx — Homme + femme - CMMx — Deux hommes (comedy)

Le joueur est toujours appelé PARTNER dans les dialogues.

■■ Le format exact des fichiers .cse n'a pas été vérifié sur des fichiers réels (section .data compressée dans le binaire).

19. Arbres & Décorations

14.1 Dualité du Système d'Arbres

SimGolf a deux systèmes d'arbres qui coexistent :

Aspect	Tuile Woods (ID 14)	Sprite FLC animé
Nature	Texture de sol (terrain type)	Objet décoratif 3D overlay
Rendu	Dans le tile grid	Par-dessus (renderObjects)
Collision balle	Atterrit dans le rough	Rebondit (obstacle solide)
Pathfinding	Coût élevé (contournable)	Infranchissable
Animé	Non (texture statique)	Oui (FLC, ~200 fichiers)
Directions	N/A	8 directions (billboard)
Usage	Grandes zones boisées	Arbres individuels

14.2 Tuile Woods (ID 14, 36 textures par thème)

```
// Propriétés
type: TileType = 14 (Tree)
textureName: "Woods" // dans les fichiers
groups: A-D (4 groupes géométriques)
cosmetics: 9 variations par groupe
totalPerTheme: 36 textures
family: grass (avec ROUGH, BUSH, FLOWER)
```

```
// Comportement
pathfinding: cost = 10 (élevé, contourné)
ball: atterrit comme dans le Rough (facteur d'atténuation)
borders: NONE (même famille que grass)
```

Arbres FLC par thème :

```
Trees/Desert/    → Palmiers, cactus, buissons secs
Trees/Links/     → Pins, bruyère, genêts
Trees/Parkland/  → Chênes, érables, buissons feuillus
Trees/Tropical/  → Palmiers tropicaux, bananiers, fougères
```

14.3 Fleurs / Buissons (ID 15)

```
// Textures: FlowerBed
// FLC: ~50 animations
// Pathfinding: Traversable
// Balle: Traversé (pas de rebond)
```

14.4 Rendu des Arbres FLC

Les arbres FLC sont **billboardés** (toujours face à la caméra) et rendus via `renderObjects()` :

```
Terrain::render()
■■■ renderTile() × N (terrain)
■■■ renderObjects() ← arbres, bâtiments, drapeaux
■   ■■■ glBindTexture(textureID)
■   ■■■ glBegin(GL_TRIANGLE_STRIP)
■   ■■■ glTexCoord2f + glVertex3f × 4
■   ■■■ glEnd()
■■■ renderPaths() (chemins)
■■■ SwapBuffers()
```

14.5 Interaction Balle/Arbres

- **Tuile Woods** : La balle atterrit dans le rough (facteur d'atténuation, pas de rebond)
- **Sprite FLC** : Rebond possible sur le tronc (coefficient = 0.40)
- **IA** : Peut choisir un **Low Shot** (trajectoire roulée sous les branches) si des arbres obstruent le chemin aérien
- **Imagination** : L'attribut influence la capacité à contourner les arbres

20. Guide de Réimplémentation (condensé)

15.1 Architecture Recommandée

```
Phaser 4 / WebGL 2.0 ou Godot 4.x
■■■ Scene principale
■   ■■■ Tilemap Layer (base) – textures de terrain
■   ■   ■■■ Tuiles isométriques (diamants, 2:1)
■   ■   ■■■ Auto-tiling par voisinage (4 sous-tuiles)
■   ■■■ Decoration Layer – sprites billboardés
■   ■■■ Path Layer – splines Bézier/Cardinal
■   ■■■ Entity Layer – golfeurs, balles
■■■ Camera (Pan, Zoom, Rotation 0/90/180/270)
■■■ UI (Canvas) – 100% custom paint
```

15.2 Priorités d'Implémentation

Priorité	Système	Complexité	Dépend de
■ P0	Terrain (grille + rendu isométrique)	Haute	—
■ P0	Auto-tiling + textures	Moyenne	Terrain
■ P0	Économie	Facile	—
■ P1	IA golfeur (sélection club + A*)	Haute	Terrain
■ P1	Physique balle	Haute	Terrain
■ P1	UI écrans + outils	Moyenne	—
■ P2	Tournois + SGA	Moyenne	Économie
■ P2	Animations FLC	Facile	Rendu
■ P2	Audio	Facile	—
■ P3	Scénarios + campagne	Moyenne	Tous

15.3 Code Clé : Interpolation de Hauteur

```
function interpolateHeight(tile: Tile, localX: number, localY: number): number {
  const el = tile.elevation; // [TL, TR, BR, BL]
  const b0 = Math.abs(el[0] - el[2]) < Math.abs(el[1] - el[3]) ? 0 : 1;
  let h: number;
  if (b0 === 0) {
    if (localX + localY > 1) {
      h = (localX+localY-1)*el[2] + (1-localX)*el[3] + (1-localY)*el[1];
    } else {
      h = (1-localX-localY)*el[0] + localX*el[1] + localY*el[3];
    }
  } else {
    // Rotation 90°
    const lx_ = localY, ly_ = 1 - localX;
    if (lx_ + ly_ > 1) {
      h = (lx_+ly_-1)*el[2] + (1-lx_)*el[1] + (1-ly_)*el[3];
    } else {
      h = (1-lx_-ly_)*el[0] + lx_*el[1] + ly_*el[3];
    }
  }
  return h;
}
```

15.4 Code Clé : Auto-Tiling (Sous-Tuiles)

```
function getSubTile(tileMap, col, row, numTypes) {
  const tileType = tileMap.get(col/2, row/2);
  const subX = col % 2, subY = row % 2;

  if (tileType == 0) return (subX + 2) + (subY + 2) * numTypes;

  const sameH = (tileMap.get(col/2 + (subX*2-1), row/2) == tileType);
  const sameV = (tileMap.get(col/2, row/2 + (subY*2-1)) == tileType);
  const sameD = (tileMap.get(col/2 + (subX*2-1), row/2 + (subY*2-1)) == tileType);

  if (sameV && sameH && sameD) return subX + subY * numTypes; // Full
  if (sameV && sameH) return (subX+2) + subY * numTypes; // Side
  if (sameH) return (subX+6) + subY * numTypes; // Outer
  if (sameV) return subX + (subY+2) * numTypes; // Inner
  return (subX+4) + subY * numTypes; // Corner
}
```

15.5 Leçons de projets externes

Open Golf Tycoon (Kenny Johnson, Godot 4.6, MIT) : - ■ Dispersion angulaire > dispersion circulaire pour réalisme - ■ Architecture événementielle (EventBus, ~60 signaux) - ■ Overlay pattern pour le terrain - ■ **Coûts linéaires cassent l'économie** → utiliser $\sqrt{\text{raw_cost}} \times 20$ - ■ Tournois factices (RNG simple sans simulation réelle) → à éviter - ■ Pas d'audio → intégrer tôt

MadcoreTom (WebGL2 + TypeScript, 2024) : - ■ Algorithme de triangle layout (split selon diagonale la plus plate) - ■ Logique de sous-tuiles par voisinage (S/in/out/full) - ■ Interpolation barycentrique pour positionnement - ■ Picking via FBO (RGB-position encoding) - ■ Texture 3D WebGL2 pour rendu multi-types

15.6 Dimensions et Constantes Clés

Élément	Valeur
Taille grille	64×64 (typique), jusqu'à 256×256
Taille tuile	584 bytes = 0x248
TILE_W	128 px
TILE_H	64 px
Élévation	0-4 (5 niveaux)
Types de terrain	16 (0-15)
Thèmes	4 (Parkland, Links, Desert, Tropical)
Résolution	800×600 minimum
Slots simulation	16 (stride 0x58)
Clubs	14 (Driver→Putter)
Skills golfeur	5-8 (selon source : putting, power, accuracy, imagination, timing, luck)
Vitesses simulation	5s, 10s, 20s (défaut), 30s entre tirs

21. Lacunes & Travaux Futurs — Bilan Résolu (Mai 2026)

Mise à jour après analyse de 42 fichiers décompilés (golf.exe + Terrain.dll + sound.dll + jgld.dll) Statut : ■ Résolu / ■■ Partiel / ■ Non résolu / ■ Nouveau gap

16.1 Résolutions Apportées par les Fichiers Décompilés

#	Lacune	Statut	Fichier(s) de résolution	Ce qui reste
1	vtable[0x68] — Simulation golfeur	■	game_vtable_dispatch.h + game_tick.c + game_advance_sim.c (290L) + game_start_action.c (266L) Ball_MainFunction @ 0x460cf0 — 362 Ko, 7 358 insn, 11 sous-fonctions décompilées. Physics_Step, Ball_Integrate, Ground_HeightAt, Wind_Calculate, CalcDistance_FPU. 19+13 FPU	Impossible sans exécution réelle (dispatch dynamique)
2	Formules physiques balle	■ Résolu	identifiées. Modèle complet traînée+Magnus+gravité+vent+rebond	Calibration des constantes (DRAG, LIFT, BOUNCE) contre comportement réel du jeu
3	Sélection de club	■	game_ai_logic.h (573L) — AI_SelectClub, AI_ChooseShotType, AI_GenerateComment	Coûts exacts de pathfinding (déduits, pas ASM)
4	Sauvegarde .sve	■	—	Pas de fichier .sve réel dans le dump
5	Structure Tile (536/584 → 568/584 connus)	■■	tile_struct.h (214L) — 12 nouveaux champs : tileFlags@0x34, variation@0x38, hasPath@0x3c, pathType@0x40, pathDirection@0x44, pathConnections@0x48	RenderPass (0x06c-0x233) mal documenté, unknown[12]@0x060, unknown[16]@0x238
6	Sous-objets C++ embarqués	■	—	Constructeurs @ 0x1000f6e0, 0x1000f7a0 non décompilés
7	TexturesSlots A/B	■■	tile_struct.h — offsets maintenant dans zone renderPasses	Structure RenderPass incomplète
8	Scénarios (.cse)	■	game_scenarios.c (191L) — format 336 bytes complet, 6 scénarios documentés	Non vérifié sur fichier réel .cse
9	Système de météo	■■	game_economy.h + game_advance_sim.c — isRaining, paramètres vent	Algorithme météo complet (WindState connu mais génération aléatoire inconnue)
10	IA sociale golfeurs	■■	game_ai_logic.h — attributs personnalité documentés	Logique sociale exacte non extraite (charisma, ambition, friendship, humor, patience, generosity)
11	Cutscenes Bink Vidéo	■	—	Fichiers .bik manquants (CD nécessaire)

16.2 Lacunes Mineures ■

#	Lacune	Statut	Notes
12	Jouabilité multi-joueur	■	Non documenté
13	Fichiers .txt de thème	■	Non trouvés dans le dump
14	Mode spectateur	■	Non documenté
15	Fichiers MIDI (musique)	■	sound_manager.c documente MIDI_Device mais pas les fichiers
16	Voix des golfeurs (WAV)	■■	Sons d'interface (21 WAV) identifiés dans game_physics.c — voix célébrités manquantes

16.3 Nouveaux Blind Spots Découverts ■

#	Lacune	Fichier source	Adresse(s)	Priorité
---	--------	----------------	------------	----------

17	Ball_MainFunction (trajectoire 3D réelle, 40+ refs FPU)	golf.exe	0x460cf0	■
18	renderPostProcess / renderEffects / renderOverlay (appelés depuis renderSingleTile)	Terrain.dll	terrain_render_tile.c	■
19	Structure RenderPass — 54/56 bytes inconnus	Terrain.dll	tile_struct.h :: RenderPass	■
20	Economy_TickYearly (1907 octets) — calcul annuel complet	golf.exe	0x49dab0	■
21	Tourney_ResultCalc — calcul exact scores tournoi	golf.exe	~0x4650ee	■
22	tileHit (picking écran→grille)	Terrain.dll	0x1000ab30	■
23	jgld.dll complet (~1200 fonctions)	jgld.dll	N/A (debug build)	■
24	SmoothInterpolator (tweening UI)	golf.exe	0x4969e0	■
25	Système sérialisation (LoadGame/SaveGame)	golf.exe	0x4a589f, 0x4a5bbd	■
26	Gestion événements clavier (mapping touches→actions)	golf.exe	game_input_handler.c	■
27	Système dialogues (fichiers .txt UTF-16LE, 90+ fichiers)	golf.exe	Chaînes @ 0x4d2730	■
28	FOV exact selon résolution (interpolation entre 3 valeurs connues)	Terrain.dll	0x10070a10	■
29	Algorithme placement arbres (renderObjects)	Terrain.dll	terrain_render_tile.c	■

16.4 Comment Débloquer les Lacunes Restantes

Méthode	Applicable à	Effort
1. Compiler GhidraCliBridge.java → décompilation headless fonctionnelle	#17, #18, #20, #21, #22, #25	30 min
2. Exécuter le jeu (tracer vtable[0x68])	#1 (dispatch dynamique)	2h (setup Wine/VM)
3. Analyse dynamique (x64dbg)	#1, #17 (trajectoire exacte)	4h
4. Trouver fichiers .sve réels	#4	30 min (recherche web)
5. Parser .cse réel	#8 (validation)	30 min
6. Ghidra → décompilation manuelle	#19 (RenderPass), #24 (Interpolator)	2h
7. Analyse jgld.dll	#23	8h (debug build, 1200 fns)

22. Annexes

17.1 Carte des Fonctions ASM — golf.exe

Point d'Entrée et Initialisation :

Adresse	Nom	Rôle	Taille
0x4a682f	WinMain	Point d'entrée CRT	164 insn
0x4a794d	HeapSetup	Configuration mémoire	—
0x4a8dec	SystemCheck	Vérification OS + résolution	—
0x4a93ff	InitGameSystems	Init tous sous-systèmes	1927 insn
0x4a50db	InitGameState	Chargement données + UI	1894 insn
0x4a5108	GameLoopCallback	Update + Render chaque frame	1876 insn
0x4a5119	GameLoopExit	Handler de sortie	—
0x4aad6a	CleanupAndExit	Nettoyage et sortie	—

Simulation et Boucle de Jeu :

Adresse	Nom	Rôle	Taille
0x49846c	GameTickFunction	Boucle simulation 16 slots	~800 insn
0x49ab40	AdvanceSimulation	Avancement timer	—
0x49acf0	StartGolferAction	Démarrage tour golfeur	2336 octets
0x49b7b0	InputHandler	Gestion des entrées (6 types)	~400 insn
0x49b690	HitTest	Collision coordonnées → tuile	~200 insn
0x497d10	ValidateAndAdvance	Transition d'état	—

Physique :

Adresse	Nom	Rôle
---------	-----	------

0x460cf0	Ball_MainFunction	Trajectoire 3D balle (40+ refs à 0x81ca10)
0x464ee0	CalcDistance_FPU	distance = property_byte × 0.05
0x474440	BallInit	Initialise structure balle (88 bytes)
0x49f370	AI_ProcessTick	Tick IA golfeur (1577 octets)
0x49bec0	AI_ChooseClub	Choix du club (196 octets)
0x49f9a0	AI_CalculateShot	Calcul du tir optimal (236 octets)
0x4a3be0	AI_EvaluateShot	Évaluation du tir par score (808 octets)

Économie :

Adresse	Nom	Rôle
0x49d820	Economy_CalculateProfit	Calcul profit journalier
0x49e450	Economy_TickDaily	Tick économique quotidien (95 octets)
0x49dab0	Economy_TickYearly	Tick économique annuel (1907 octets)

Tournois :

Adresse	Nom	Rôle
0x484e30	Tourney_InitObj	Initialise un objet tournoi
0x4481b0	Tourney_Setup	Configure avant tournoi
0x448220	Tourney_InitAll	Init tous les objets
0x447000 - 0x4480f7	Tourney_UICreate	Panel UI tournoi (4300 insn)

I/O et Sauvegarde :

Adresse	Nom	Rôle
0x4a5d5f	IO_OpenFile	CreateFileA (719 octets)
0x4a589f	IO_ReadFile	ReadFile (473 octets)
0x4a5bbd	IO_WriteFile	WriteFile (395 octets)
0x4a5a78	IO_CloseFile	CloseHandle (93 octets)
0x4a64b8	DeleteSaveFile	DeleteFileA
0x4a9620	SeekInFile	SetFilePointer

17.2 Carte des Fonctions — Terrain.dll (base 0x10000000)

Adresse	Export (nettoyé)	Rôle
0x10001d50	tileAt(x, y)	Accès tuile row-major
0x10001de0	getElevation(tile, corner)	Hauteur d'un coin
0x10001f10	getType(tile)	Type de terrain
0x10001e80	getWall(tile, side)	Mur côté
0x1000a560	setWall(tile, type, side, bool)	Pose mur
0x1000a510	hasPath(tile)	Vérifie chemin
0x1000a450	hasConnectedPath(x, y)	Connectivité chemin
0x100031a0	getInstance()	Singleton Terrain
0x10005990	render(tile, height)	Rendu principal isométrique
0x100011ea	renderTile(x, y, w, h, flags)	Rendu tuile simplifié
0x1000e6c0	renderSingleTile(tile)	Rendu multi-passes OpenGL
0x100048a0	drawLine(x1,y1,x2,y2,color,w)	Bresenham isométrique
0x100042a0	drawCircle(tile, radius)	Cercle isométrique
0x10005230	drawBezierSpline(...)	Bézier cubique pour chemins
0x10009c80	initSystem(w, h, hDC, fs)	Init moteur + alloc tuiles
0x10009ed0	resize(w, h)	Redimensionner grille
0x1000aa10	resetTerrain()	Init toutes les tuiles (eau)
0x1000c2c0	tileInit(tile, row, col)	Init une tuile
0x10001af0	loadNewCourseType(type)	Change le thème
0x100032f0	setType(tile, type, variation)	Définit type + variation
0x10003390	getVariation(tile)	Variation cosmétique
0x1000a5c0	elevateCorner(tile, corner)	Élève un coin
0x1000a620	lowerCorner(tile, corner)	Abaisse un coin
0x1000a6f0	calcNormals(tile)	Normale d'une tuile
0x1000a740	calcAllNormals(tile)	Recalcul toutes normales
0x10008fc0	setZoomLevel(level)	Niveau de zoom
0x100011c2	changeLighting(level)	Change éclairage
0x1000ab30	tileHit(x, y)	Picking écran → tuile
0x1000a840	setSplineHeight(height)	Hauteur spline

17.3 Variables Globales

Adresse	Variable	Type
0x008400b0	g_pGameObject	GameState*
0x00842160	g_hInstance	HANDLE

0x00840aac	g_hWnd	HWND
0x00842040	g_initPool	InitSlot[32]
0x0080d840	g_tournaments	TournamentObj[22]
0x00840aa4	g_gameFlags	int32
0x0083c340	g_statusBuf	char[256]
0x0083afcc	g_closeFlag	int32
0x81ca10	g_ballData	BallState[]
0x4ba818	CONST_0.05	double
0x10070a0c	g_courseTheme	int32
0x100687f8	g_textureTable	GLuint[]

17.4 Chaînes Importantes dans le Binaire

Adresse	Chaîne
0x4d0c88	"SGA Qualifying School"
0x4d0d6c	"Grand Slam Championship!"
0x4c6144	"Begin Tournament"
0x4c5b00	"SGA Evaluation"
0x4c3cf0	"Cannot save course during a tournament."
0x4cff90	"TOURNAMENT RESULTS"
0x4d15c4	"LEADER BOARD of"
0x4bf408	"1st Tournament"
0x4c5328	Themes\\Championship\\
0x4c8944	Themes\\Championship*.cse
0x4c6614	Themes\\Championship*.pro
0x4c6c54	"Sandbox Mode"
0x4c6c64	"Play a Championship"
0x4c5a48	"for Championship"

17.5 Références Externes

Open Golf Tycoon (Kenny Johnson, Godot 4.6, MIT) - Blog : <https://kennyatx.com/vibecoding-a-golf-course-builder-on-parental-leave-lessons-learned-and-an-open-beta/> - Repo : <https://github.com/sneesh/opengolf-tycoon> - ~25,000+ lignes GDScript, 150+ fichiers - Coût total : ~\$140 (Claude Max + compute) - Leçons clés : dispersion angulaire, sub-linear cost scaling, architecture événementielle

MadcoreTom — Heightmaps and Golf (WebGL2 + TypeScript, 2024) - Site : <https://www.madcoretom.com/p/heightmaps/index.html> - Code source complet embarqué en JS (ESBuild bundle) - Référence la plus proche du rendu original SimGolf - Techniques : triangle layout, sous-tuiles 2x2, texture 3D, picking FBO, VBO dynamique

17.6 Convertisseur FLC → Animated WebP

Pipeline :

decode_flg.py → Décodeur FLC complet (opcodes : SS2, BRUN, COLOR, LC)

- Palette externe auto-découverte
- Chroma key (magenta 255,0,255 + index 0 → transparent)
- Gestion des frames delta
- Export Animated WebP

convert_all_to_webp.py → Convertisseur unifié :

- PCX / BMP → WebP lossless
- FLC → Animated WebP
- Extraction tuiles 64x64 des atlas

Résultat : 1893 fichiers → Animated WebP (34 Mo vs 588 Mo en PNG, -94%) **Assets convertis** : Tous disponibles dans data/converted/

17.7 Outils de la Chaîne de RE

Outil	Usage
objdump + capstone	Désassemblage massif (1.1M lignes)
strings	Extraction de chaînes
Ghidra 12.1	Décompilation C — 42 fichiers extraits (2900+ fonctions)
pefile	Analyse PE
hexdump / xxd	Analyse binaire
Python struct	Analyse de données binaires
ghidra CLI (Rust)	Interface ligne de commande pour Ghidra

17.8 Index des 42 Fichiers Décompilés

golf.exe (28 fichiers) : | Fichier | Lignes | Contenu | |-----|:-----|:-----| | game_advance_sim.c | 290 | Simulation avancée (timing, transitions) | | game_ai_logic.h | 573 | IA golfeur : sélection club, type tir, commentaires | | game_coursengine.c | 321 | Moteur de parcours (validations, transitions) | | game_data_parser.h | 640 | Parser données golfeurs (.dta, .pro) | | game_data_types.h | 210 | Types de données communs | | game_economy.c | 371 | Calculs économiques (revenus, coûts, profit) | | game_economy.h | 273 | Structures économiques | | game_init_audio.c | 176 | Initialisation audio | | game_init_state.c | 245 | Initialisation état de jeu | | game_init_systems.c | 247 | Initialisation sous-systèmes | | game_input_handler.c | 259 | Gestion des entrées (6 types d'événements) | | game_loop.c | 240 | Boucle de jeu principale | | game_misc.c | 185 | Fonctions diverses (utilitaires) | | game_physics.c | 302 | Physique de balle (trajectoire, vent, lie) | | game_scenarios.c | 191 | Système de scénarios et campagnes | | game_scoring_sga.c | 348 | Évaluation SGA, scoring, stats | | game_start_action.c | 266 | Démarrage action golfeur (vtable dispatch) | | game_tick.c | 432 | Boucle simulation 16 slots | | game_tick_v2.c | 573 | Version alternative tick (optimisée) | | game_tilegrid.c | 262 | Gestion grille de tuiles | | game_tournaments.c | 935 | Système complet de tournois (14 types, leaderboard) | | game_ui_statemachine.h | 262 | Machine d'état UI (25 états) | | game_ui_system.c | 941 | Système UI complet | | game_vtable_dispatch.h | 90 | Analyse vtables C++, dispatch dynamique | | game_winmain.c | 302 | Point d'entrée Windows (WinMain) | | smooth_interpolator.c | 284 | Interpolateur fluide (tweening UI) | | sound_manager.c | 566 | Décompilation complète sound.dll (12 exports) | | test_game_data_parser.c | — | Tests unitaires parser |

Terrain.dll (14 fichiers) : | Fichier | Lignes | Contenu | |-----|:-----|:-----| | terrain_drawing.c | 265 | Primitives dessin (lignes, cercles isométriques) | | terrain_elevation.c | 168 | Gestion élévation (elevate/lower corner) | | terrain_initSystem.c | 228 | Initialisation OpenGL + allocation grille | | terrain_normals.c | 296 | Calcul normales pour éclairage | | terrain_paths.c | 307 | Chemins (Bezier, Cardinal, pathfinding) | | terrain_render.c | 276 | Pipeline rendu principal (full/focus) | | terrain_render_helpers.c | — | Helpers rendu (coordonnées, culling) | | terrain_render_tile.c | — | Rendu individuel tuile (multi-passes) | | terrain_setType.c | 228 | Définition type terrain + variation | | terrain_tileAt.c | — | Accès tuile dans la grille | | terrain_zoom.c | 240 | Gestion zoom caméra | | tile_getters.c | — | Getters Tile (type, elevation, walls) | | tile_struct.h | 214 | Structure Tile complète (584 bytes) |

jpgld.dll (1 fichier) : | Fichier | Lignes | Contenu | |-----|:-----|:-----| | jpgld_engine.c | 414 | Moteur 2D GDI (JackalClass, ~1200 fonctions) |

23. Pipeline d'Assets — Analyse Complète

Analyse basée sur : 42 fichiers décompilés + 6 scripts de conversion + structure des dossiers data/raw/ **Confiance :** ■ Confirmé (données extraites) | ■■ Reconstruit (code C déduit) | ■ Non extrait

23.1 Architecture Globale des Assets

```
simgolf-re/data/raw/
■■■■ Data/ (589 fichiers) – Textures de terrain BMP + atlas PCX
■ ■■■■ Textures/ – Textures BMP par thème (desert/parkland/links/tropical)
■ ■■■■ {theme}.pcx – Atlas de textures (planches 64×64)
■ ■■■■ {Type}{Groupe}{Variation}.bmp – Textures individuelles
■■■■ Flics/ (1 892 fichiers) – Animations FLC
■ ■■■■ Male/ (~500) – Golfeurs (30 animations × 8 directions)
■ ■■■■ Female/ (~400) – Golfeuses
■ ■■■■ Trees/ (~200) – 4 thèmes × espèces
■ ■■■■ Bldgs/ (~100) – Bâtiments
■ ■■■■ Homes/ (~100) – Maisons
■ ■■■■ Flowers/ (~50) – Fleurs
■ ■■■■ Water/ (~50) – Eau animée
■ ■■■■ Animals/ (~30) – Animaux
■ ■■■■ Celebs/ (~30) – Célébrités
■ ■■■■ Employee/ (~30) – Employés
■ ■■■■ Bridges/ (~30) – Ponts
■ ■■■■ Scenic/ (~50) – Décorations
■ ■■■■ Tees/ (~20) – Marqueurs de départ
■■■■ Interface/ (646 fichiers) – UI sprites PCX
■ ■■■■ {Name}.pcx / {Name}_A.pcx – Sprite normal + alpha mask
■ ■■■■ Panels/ – Barres d'outils (5 panels × 2)
■ ■■■■ Buttons/ – Boutons d'interface
■ ■■■■ Popups/ – Fenêtres modales
■■■■ Themes/ (5 dossiers) – Données de thème
■ ■■■■ Championship/ – Scénarios (.cse, .pro)
■ ■■■■ Standard/ – Textures de base
■ ■■■■ Firaxis/ More_Stories/ The_Sims/ – Packs additionnels
■■■■ Sounds/ (21+ WAV) – Audio
■ ■■■■ Golf sfx/ – Sons de golf (Drive, Putt, Chip, etc.)
■ ■■■■ Effects/ – Effets (vent, eau, oiseaux)
■ ■■■■ Celebs/ – Voix célébrités
■ ■■■■ Emotion/ – Émotions des golfeurs
■■■■ Bodies/ – Corps des golfeurs (sprites)
■■■■ Heads/ – Têtes des golfeurs (portraits)
■■■■ *.bik – Cutscenes Bink Video (2 fichiers)
```

23.2 Le Format FLC (Animations)

Header Custom SimGolf (128 bytes)

```
# decode_flg.py - FLCDecoder._parse_header()
# Offset  Champ      Type      Description
# 0x00    file_size   uint32    Taille totale du fichier
# 0x04    magic        uint16    Toujours 0xAF12 (FLC standard)
# 0x06    frames_count uint16    Nombre de frames
# 0x08    width        uint16    Largeur en pixels
# 0x0A    height       uint16    Hauteur en pixels
# 0x0C    depth        uint16    Toujours 8 (256 couleurs)
# 0x0E    flags        uint16    Drapeaux
# 0x10    speed        uint32    ms par frame (delay)
# 0x14-0x7F padding    -          Réservé (128 bytes total)
# 0x80+   frame_data   -          Données de frames
```

Différence avec FLC standard : le header custom SimGolf a **4 bytes supplémentaires au début** (file_size), décalant le header FLC standard de +4 bytes. Les données de frame commencent toujours à 0x80.

Structure d'une Frame

```
# Chaque frame commence par :
# offset 0: frame_size   uint32    Taille de cette frame
# offset 4: magic        uint16    0xF1FA (FLC frame magic)
# offset 6: chunks_count uint16    Nombre de chunks dans cette frame
# offset 8: padding[8]   -          Réservé
# offset 16: chunk_data[] -         Données des chunks

# Chaque chunk :
# offset 0: chunk_size   uint32    Taille de ce chunk
# offset 4: chunk_type   uint16    Type d'opcode
# offset 6: payload      -          Données du chunk
```

Opcodes Supportés (5 types)

Opcode	Hex	Nom	Description
0x07	0x0B → corrigé	FLI_SS2	Delta word-oriented (2-px columns) — compression par paires de colonnes
0x0B	0x0B	FLI_COLOR	Mise à jour palette (256 couleurs, format RGB 6-bit → 8-bit)
0x0C	0x0C	FLI_LC	Delta byte-oriented (compression par ligne)
0x0D	0x0D	FLI_BLACK	Frame noire (reset pixels à 0)
0x0F	0x0F	FLI_BRUN	Byte RLE (run-length encoding plat, pas de lignes)
0x10	0x10	FLI_COPY	Keyframe (copie brute de tous les pixels)

Palette System

```
# FLI_COLOR decode:
def _decode_palette(self, data):
    packet_count = struct.unpack('<H', data[0:2])[0] # nombre de packets
    pos = 2
    for _ in range(packet_count):
        skip = data[pos] # index de début (0-255)
        count = data[pos + 1] # nombre de couleurs (0 = 256)
        pos += 2
        for j in range(count):
            # RGB 6-bit → 8-bit (décalage)
            r = (data[pos] << 2) | (data[pos] >> 6)
            g = (data[pos+1] << 2) | (data[pos+1] >> 6)
            b = (data[pos+2] << 2) | (data[pos+2] >> 6)
            palette[skip + j] = (r, g, b)
            pos += 3
```

Les palettes initiales sont stockées dans des fichiers .pcx dédiés (ex: Flag_DESERTpal.pcx). Le décodeur FLC les découvre automatiquement par : 1. Correspondance exacte : Pal{BaseName}.pcx 2. Correspondance par mot-clé : {BaseName}Pal*.pcx 3. Correspondance par mots communs (≥3 chars) 4. Dossier partagé : *Pal*.pcx, *_palette.pcx, *.pal

Chroma key : - Index 0 (noir) → transparent - Magenta (255,0,255) → transparent

Golfeur Animations (30 types × 8 directions)

Les golfeurs ont 30 animations, chacune dans 8 directions (0°-315°) :

```
animations = {
    'NormalAddress', 'NormalSwing', 'NormalFollowThru',
    'NormalWalk', 'TiredWalk',
    'PuttAddress', 'PuttSwing', 'PuttFollowThru', 'LineUpPutt',
    'PitchSwing', 'PitchFollowThru',
    'SandSwing', 'SandFollowThru',
    'SuccessA', 'SuccessB', 'Sad',
    'Sitting', 'LeanLeft', 'LeanRight',
    'LookAtShot', 'TipHat',
    'ThrowClub', 'KickBag',
    'Shadow'
}
```

Convention de nommage :

Male{NomAnimation}_000.flc → Direction 0° (face caméra)

Male{NomAnimation}_045.flc → Direction 45°

... jusqu'à 315° (8 directions)

Tailles des sprites :

Golfeur : 64×130 px

Arbre : 48×96 px

Fleur : 32×32 px

Bâtiment: 96×128 px

23.3 Le Format PCX (UI, Portraits, Palettes)

Format PCX standard (ZSoft PCX, v3/v5) avec : - Palette 256 couleurs en fin de fichier (byte 0x0C + 768 bytes RGB) - Compression RLE simple

```
# convert_all_to_webp.py - convert_image_to_webp()
# Pipeline de conversion PCX → WebP :
img = Image.open(src_path).convert('RGBA')
# Chroma key : noir (0,0,0) et magenta (255,0,255) → transparent
for r, g, b, a in data:
    if (r == 0 and g == 0 and b == 0) or (r == 255 and g == 0 and b == 255):
        new_data.append((0, 0, 0, 0)) # transparent
```

UI Sprites : Chaque sprite UI existe en deux versions : - {Nom}.pcx — Sprite normal - {Nom}_A.pcx — Masque alpha (transparence)

Le compositing utilise la technique GDI classique : SRCAND + SRCPAINT.

Portraits des golfeurs : Stockés dans les fichiers .pro / .chr après le marqueur *PCXFILE (offset 0x722). Format PCX standard.

23.4 Atlas de Textures (PCX → BMP tuiles)

Les textures de terrain sont stockées dans des **atlas PCX** :

Data/parkland.pcx	(512×512 px) → 64×64 = 64 tuiles
Data/desert.pcx	(512×512 px)
Data/links.pcx	(512×512 px)
Data/tropical.pcx	(512×512 px)

Chaque atlas est une grille de **tuiles 64×64 px**. Le nombre exact de tuiles varie par thème : - Desert : 732 fichiers BMP individuels (+ planche PCX) - Links : 644 - Parkland : 602 - Tropical : 547

Les tuiles individuelles sont extraites via `extract_atlas_tiles()` (découpage en 64×64). Les textures individuelles sont des .bmp nommées selon la convention {Type}{Orientation}{Variation à 4 chiffres}.bmp.

23.5 Le Moteur 2D JGL (jgld.dll)

Architecture

// jgld.dll — Jackal Graphics Library (1.2 MB Debug, 396 KB Release)

// Export unique : `get_graphsy_object_ptr()` → `JackalClass*`

JackalClass (332 bytes = 0x14C)

■■■ vtable	(pointeur vtable C++)
■■■ hWnd	(fenêtre principale)
■■■ hDC	(device context fenêtre)

```

■■■■ hSpriteDC          (DC off-screen pour sprites)
■■■■ hSpriteBMP         (DIBSection framebuffer sprites)
■■■■ hPalette           (palette 8-bit)
■■■■ screenWidth        (800+)
■■■■ screenHeight       (600+)
■■■■ bitsPerPixel       (8 ou 16)
■■■■ framebuffer*       (pointeur DIBSection)
■■■■ framebufferPitch    (stride bytes)
■■■■ gammaCorrection     (correction gamma)
■■■■ hFont              (police GDI courante)

```

Pipeline de Rendu 2D

```

// Compositing final (chaque frame) :
BitBlt(hDCWindow, dstX, dstY, w, h, hSpriteDC, srcX, srcY, SRCCOPY);

// Sprites avec transparence (alpha mask + sprite) :
// 1. Dessiner alpha mask : SRCAND (préserve fond)
// 2. Dessiner sprite : SRCPAINT (superpose)
BitBlt(hDC, x, y, w, h, hMaskDC, 0, 0, SRCAND);
BitBlt(hDC, x, y, w, h, hSpriteDC, 0, 0, SRCPAINT);

```

Sous-systèmes Identifiés

Sous-système	Fichiers source estimés	Rôle
jglmain.cpp	?	Point d'entrée, initialisation, boucle message
jglsprite.cpp	?	Rendu sprites 2D (BitBlt/StretchBlt)
jglsprite_8_16c.cpp	?	Sprites 8-bit palettisés / 16-bit high-color
jglfont.cpp	?	Polices GDI (CreateFontIndirect, TextOut)
jglpng.cpp	?	Chargement PNG (via libpng 1.0.5 intégrée)
jglsystem.cpp	?	Gestion fenêtre, résolution, palette
libpng 1.0.5	?	Bibliothèque PNG intégrée au binaire

23.6 Rendu des Sprites 3D (Overlay / Billboarding)

Les sprites 3D (arbres, golfeurs, bâtiments) sont rendus dans `renderObjects()` en OpenGL **après** le terrain :

```

// Pipeline de rendu overlay :
Terrain::render()
■■■■ renderTile() × N          // Terrain isométrique (OpenGL 3D)
■■■■ renderObjects()          // ← Sprites billboardés
■ ■■■■ glBindTexture(textureID) // Texture du sprite (FLC frame courante)
■ ■■■■ glBegin(GL_TRIANGLE_STRIP)
■ ■ ■■ glVertex3f(0,1); glVertex3f(x-W/2, y, z) // TL
■ ■ ■■ glVertex3f(0,0); glVertex3f(x-W/2, y+H, z) // BL
■ ■ ■■ glVertex3f(1,1); glVertex3f(x+W/2, y, z) // TR
■ ■ ■■ glVertex3f(1,0); glVertex3f(x+W/2, y+H, z) // BR
■ ■■■■ glEnd()
■■■■ renderPaths()            // Chemins (splines)
■■■■ SwapBuffers()            // Présentation

```

Billboarding : Les sprites sont toujours face à la caméra (dans le plan XY perpendiculaire à l'axe Z). La sélection de frame FLC dépend de l'angle de vue (parmi 8 directions :

```

// Mapping direction golfeur → fichier FLC
// angle 0° → _000.flc (face)
// angle 45° → _045.flc
// angle 90° → _090.flc
// ... jusqu'à 315°
int flcDirection = ((cameraAngle + 22) / 45) * 45; // snap to nearest 45°

```

23.7 Système Audio (sound.dll)

```

// 12 exports, 3 types de devices :
// - Wave_Device (DirectSound/WaveOut) – 16864 bytes
// - Midi_Device (WINMM midiOut) – 16472 bytes
// - WaveIn_Device (WINMM waveIn) – 76 bytes

// Sons WAV identifiés (21 fichiers) :
// Boutons UI : button1-3.wav, bass down/up, slide pop up, check box, text box
// Construction : building.wav, path.wav, bridge.wav, bench.wav, place rocks/water

```

```
// Golf : Drive With Ball.wav, Putt.wav, Chip.wav, Sand.wav, Fairway.wav, Hole.wav
// Autres : unavailable.wav, wrong.wav, violin down, test{2,7,9,10}.wav

// MIDI : fichiers non trouvés dans le dump actuel
// Voix célébrités : Sounds/Celebs/*.wav – non extraites
```

23.8 Pipeline de Conversion (Assets bruts → WebP)

raw/	converted/	
■■■■ Data/*.pcx	■■■■ webp/	← Images fixes
■ ■■■■ (atlas 512×512)	■ ■■■■ tiles/	← Tuiles 64×64 extraites
■■■■ Data/Textures/*.bmp	■ ■■■■ *.webp	← Textures individuelles
■■■■ Flics/*.flc	■■■■ animations/	← Animated WebP
■■■■ Interface/*.pcx	■ ■■■■ *.webp	← UI sprites
■■■■ Bodies/*.pcx	■	← Corps golfeurs
■■■■ Heads/*.pcx	■	← Têtes/portraits
■■■■ Sounds/*.wav	■■■■ (non converti)	← Audio natif

Résultat de la conversion : 5 736 fichiers sources → WebP (188 Mo) : - 2 671 BMP → WebP (34 Mo) - 646 PCX → WebP (85 Mo) - 1 893 FLC → animated WebP (34 Mo) vs 588 Mo en PNG (–94%) - 36 TGA → WebP (5 Mo)

23.9 Inventaire Complet des Assets

Catégorie	Format	Fichiers	Taille brute	Usage
Textures terrain	BMP + PCX	2,671 + 4 atlas	~70 Mo	Rendu isométrique 3D
UI Sprites	PCX	646	~85 Mo	Interface utilisateur
Animations golfeurs	FLC	~900	~60 Mo	30 types × 8 dir × 2 sexes
Animations arbres	FLC	~200	~15 Mo	4 thèmes × espèces
Animations bâtiments	FLC	~100	~10 Mo	Constructions
Animations eau	FLC	~50	~5 Mo	Cascades, fontaines
Décors divers	FLC	~595	~40 Mo	Fleurs, animaux, ponts
Sons WAV	WAV	21+	~2 Mo	UI, golf, ambiances
MIDI	MIDI	?	?	Musique de fond
Cutscenes	BIK	2	~50 Mo	Intros
Textures chemins	TGA	36	~5 Mo	Tarmac, sable, ponts
Polices	TTF	2	~0.5 Mo	Polices TrueType
Dialogues	TXT	90+	~1 Mo	UTF-16LE scénario
Profils golfeurs	PRO/CHR/DTA	~100	~2 Mo	Binaires + PCX portraits
Scénarios	CSE	6+	~2 KB	Binaires 336 bytes
Thèmes	Txt	~5	~1 KB	Configurations couleur